In [1]:
```
# MOVIE RATING ANALYTICS (ADVANCED VISULIZATION)

import pandas as pd
import os
```

In [2]:
```
os.getcwd() # if you want to change the working directory
```

Out[2]: 'C:\\Users\\suras'

In [3]:
```
movies = pd.read_csv(r"C:\Users\suras\OneDrive\NARESH IT\Practice tasks\Movie ratin
```

In [4]:
```
movies
```

Out[4]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million $)	Year of release
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009
...	...	...	...	...	...	...
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
556	Zodiac	Thriller	89	73	65	2007
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

559 rows × 6 columns

In [5]:
```
len(movies)
```

Out[5]: 559

In [6]:
```
movies.head()
```

Out[6]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million $)	Year of release
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

In [7]:
```
movies.tail()
```

Out[7]:

	Film	Genre	Rotten Tomatoes Ratings %	Audience Ratings %	Budget (million $)	Year of release
554	Your Highness	Comedy	26	36	50	2011
555	Youth in Revolt	Comedy	68	52	18	2009
556	Zodiac	Thriller	89	73	65	2007
557	Zombieland	Action	90	87	24	2009
558	Zookeeper	Comedy	14	42	80	2011

In [8]:
```
movies.columns
```

Out[8]:
```
Index(['Film', 'Genre', 'Rotten Tomatoes Ratings %', 'Audience Ratings %',
       'Budget (million $)', 'Year of release'],
      dtype='object')
```

In [9]:
```
movies.columns = ['Film', 'Genre', 'CriticRating', 'AudienceRating','BudgetMillions
```

In [10]:
```
movies.head() # Removed spaces & % removed noise characters
```

Out[10]:

	Film	Genre	CriticRating	AudienceRating	BudgetMillions	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

In [11]:
```
movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   Film            559 non-null    object
 1   Genre           559 non-null    object
 2   CriticRating    559 non-null    int64 
 3   AudienceRating  559 non-null    int64 
 4   BudgetMillions  559 non-null    int64 
 5   Year            559 non-null    int64 
dtypes: int64(4), object(2)
memory usage: 26.3+ KB
```

In [12]:
```
movies.describe()
```

Out[12]:

	CriticRating	AudienceRating	BudgetMillions	Year
count	559.000000	559.000000	559.000000	559.000000
mean	47.309481	58.744186	50.236136	2009.152057
std	26.413091	16.826887	48.731817	1.362632
min	0.000000	0.000000	0.000000	2007.000000
25%	25.000000	47.000000	20.000000	2008.000000
50%	46.000000	58.000000	35.000000	2009.000000
75%	70.000000	72.000000	65.000000	2010.000000
max	97.000000	96.000000	300.000000	2011.000000

In [13]:
```
movies['Film']
#movies['Audience Ratings %']
```

Out[13]:
```
0      (500) Days of Summer
1                10,000 B.C.
2                 12 Rounds
3                 127 Hours
4                 17 Again
               ...
554           Your Highness
555         Youth in Revolt
556                  Zodiac
557             Zombieland
558               Zookeeper
Name: Film, Length: 559, dtype: object
```

In [14]:
```
movies.Film
```

Out[14]:
```
0      (500) Days of Summer
1                10,000 B.C.
2                 12 Rounds
3                  127 Hours
4                  17 Again
               ...
554            Your Highness
555          Youth in Revolt
556                   Zodiac
557              Zombieland
558                Zookeeper
Name: Film, Length: 559, dtype: object
```

In [15]:
```
movies.Film = movies.Film.astype('category')
```

In [16]:
```
movies.Film
```

Out[16]:
```
0      (500) Days of Summer
1                10,000 B.C.
2                 12 Rounds
3                  127 Hours
4                  17 Again
               ...
554            Your Highness
555          Youth in Revolt
556                   Zodiac
557              Zombieland
558                Zookeeper
Name: Film, Length: 559, dtype: category
Categories (559, object): ['(500) Days of Summer ', '10,000 B.C.', '12 Rounds ',
'127 Hours', ..., 'Youth in Revolt', 'Zodiac', 'Zombieland ', 'Zookeeper']
```

In [17]:
```
movies.head()
```

Out[17]:

	Film	Genre	CriticRating	AudienceRating	BudgetMillions	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

In [18]:
```
movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Film            559 non-null    category
 1   Genre           559 non-null    object
 2   CriticRating    559 non-null    int64
 3   AudienceRating  559 non-null    int64
 4   BudgetMillions  559 non-null    int64
 5   Year            559 non-null    int64
dtypes: category(1), int64(4), object(1)
memory usage: 43.6+ KB
```

In [19]:
```
movies.Genre = movies.Genre.astype('category')
movies.Year = movies.Year.astype('category')
```

In [20]:
```
movies.Genre
```

Out[20]:
```
0         Comedy
1      Adventure
2         Action
3      Adventure
4         Comedy
          ...
554       Comedy
555       Comedy
556     Thriller
557       Action
558       Comedy
Name: Genre, Length: 559, dtype: category
Categories (7, object): ['Action', 'Adventure', 'Comedy', 'Drama', 'Horror', 'Roma
nce', 'Thriller']
```

In [21]:
```
movies.Year
```

Out[21]:
```
0      2009
1      2008
2      2009
3      2010
4      2009
       ...
554    2011
555    2009
556    2007
557    2009
558    2011
Name: Year, Length: 559, dtype: category
Categories (5, int64): [2007, 2008, 2009, 2010, 2011]
```

In [22]:
```
movies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype   
---  ------          --------------  -----   
 0   Film            559 non-null    category
 1   Genre           559 non-null    category
 2   CriticRating    559 non-null    int64   
 3   AudienceRating  559 non-null    int64   
 4   BudgetMillions  559 non-null    int64   
 5   Year            559 non-null    category
dtypes: category(3), int64(3)
memory usage: 36.5 KB
```

In [23]:
```
movies.Genre.cat.categories
```

Out[23]:
```
Index(['Action', 'Adventure', 'Comedy', 'Drama', 'Horror', 'Romance',
       'Thriller'],
      dtype='object')
```

In [24]:
```
movies.describe()
```

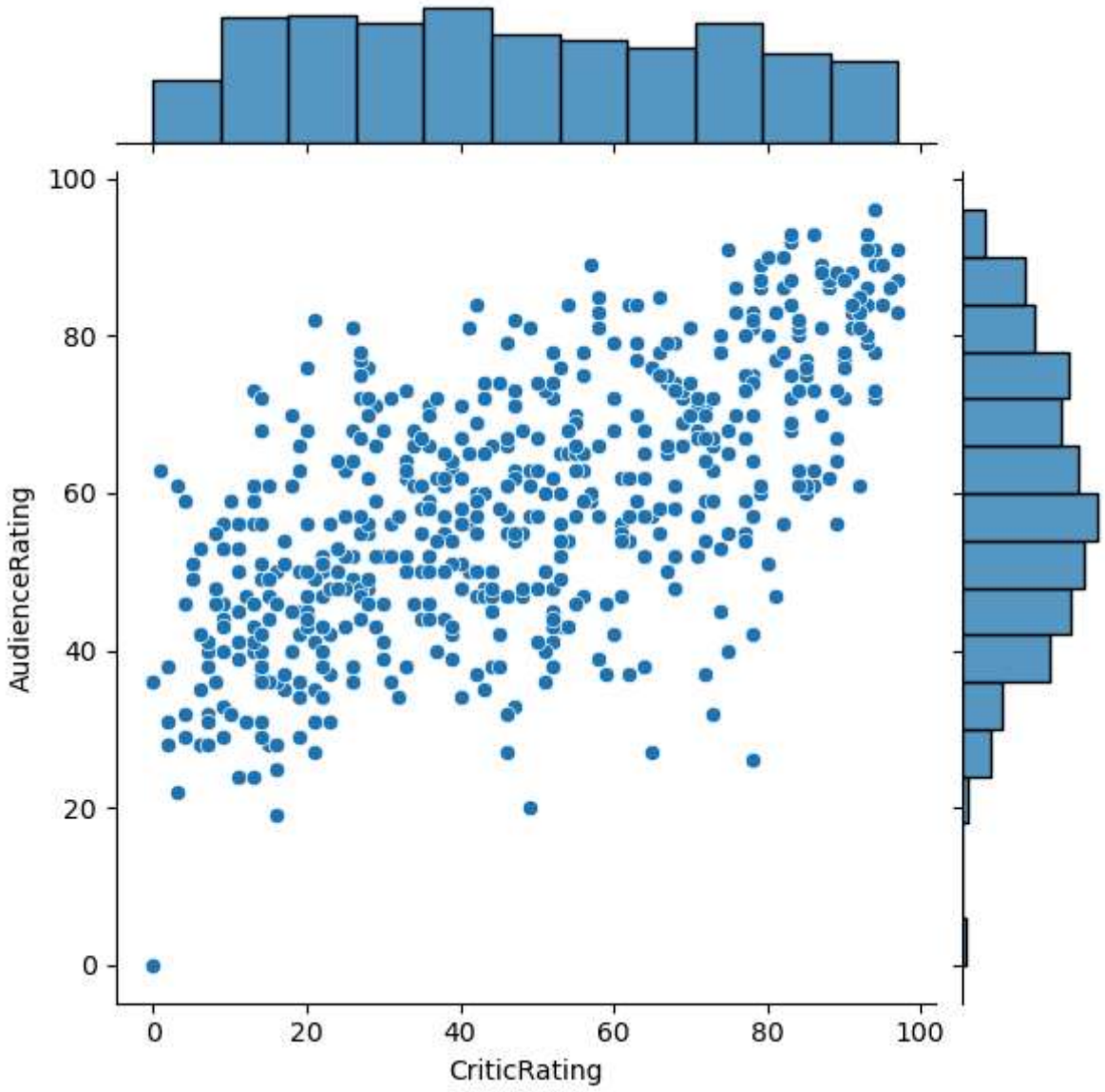
Out[24]:

	CriticRating	AudienceRating	BudgetMillions
count	559.000000	559.000000	559.000000
mean	47.309481	58.744186	50.236136
std	26.413091	16.826887	48.731817
min	0.000000	0.000000	0.000000
25%	25.000000	47.000000	20.000000
50%	46.000000	58.000000	35.000000
75%	70.000000	72.000000	65.000000
max	97.000000	96.000000	300.000000

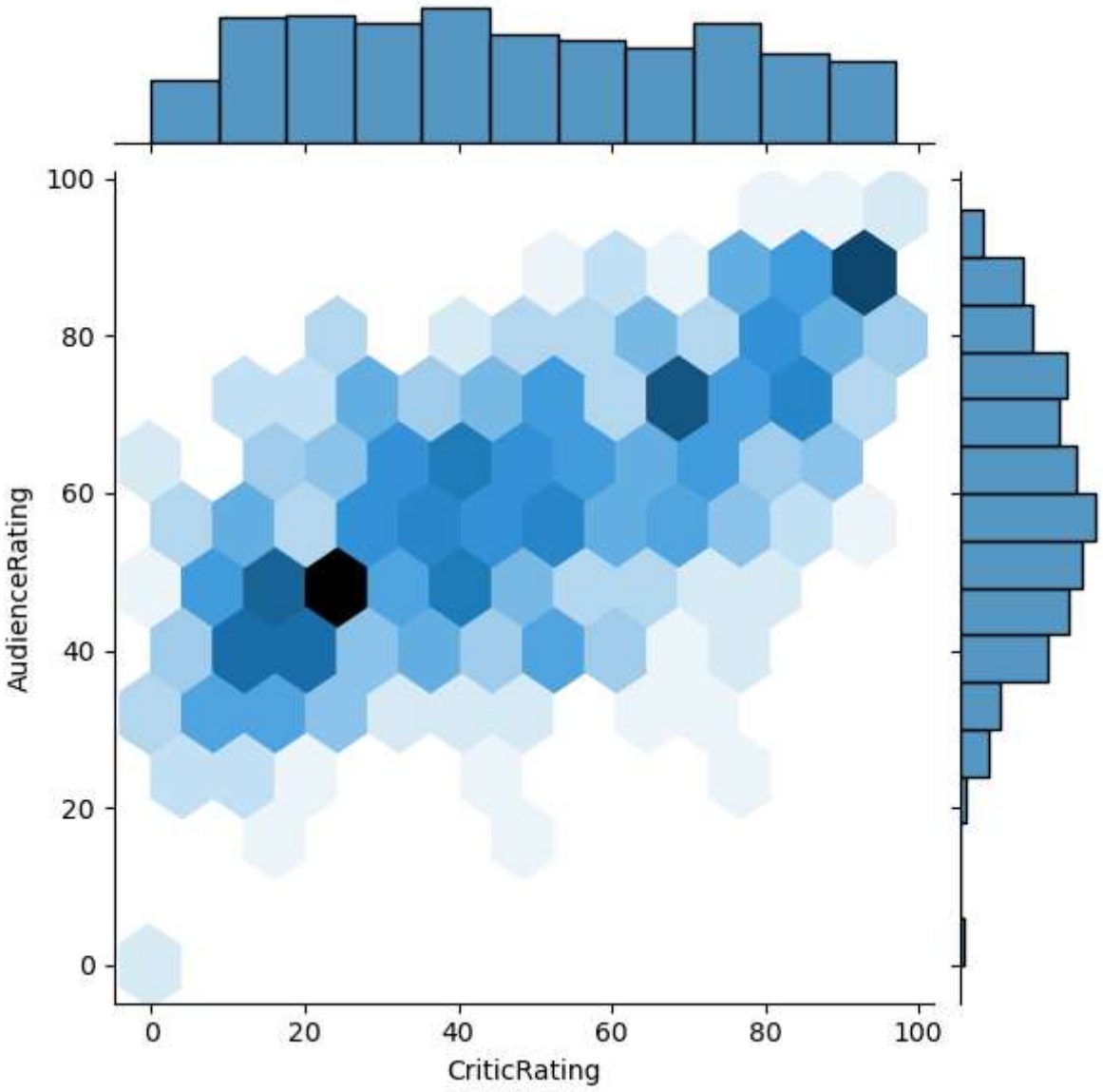
In [25]:
```
from matplotlib import pyplot as plt
import seaborn as sns
%matplotlib inline
import warnings
warnings.filterwarnings('ignore')
```

In [26]:
```
j = sns.jointplot( data = movies, x = 'CriticRating', y = 'AudienceRating')
```

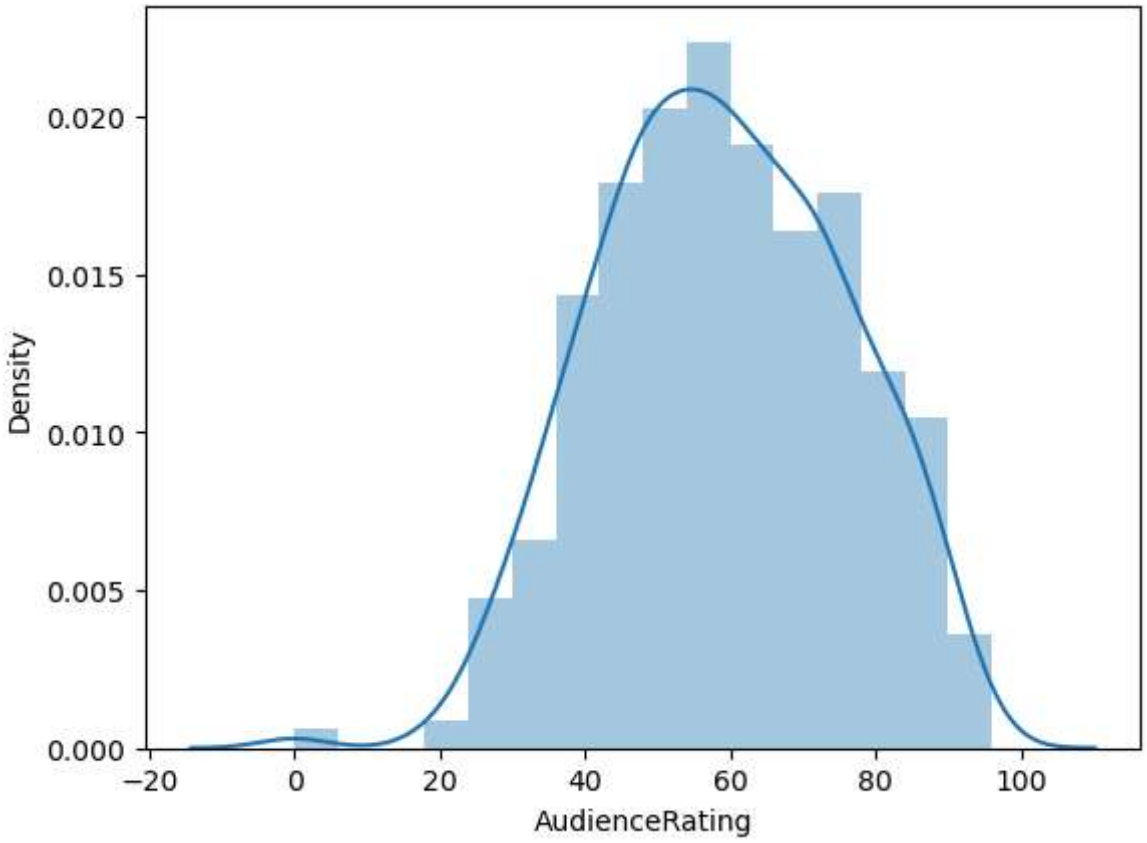
In [27]:
```
plt.show()
```



In [28]:
```
j = sns.jointplot( data = movies, x = 'CriticRating', y = 'AudienceRating', kind='h
plt.show()
```



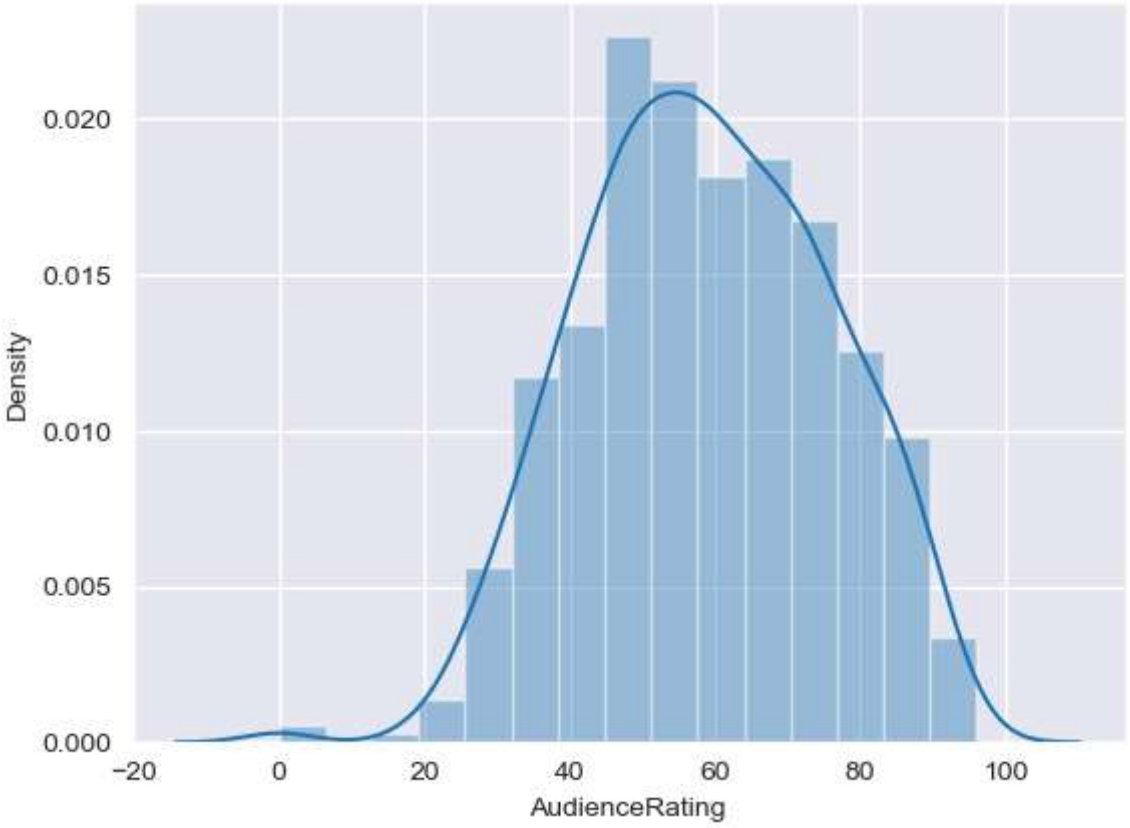
In [29]:
```
m1 = sns.distplot(movies.AudienceRating)
plt.show()
```



In [30]:
```
sns.set_style('darkgrid')
```

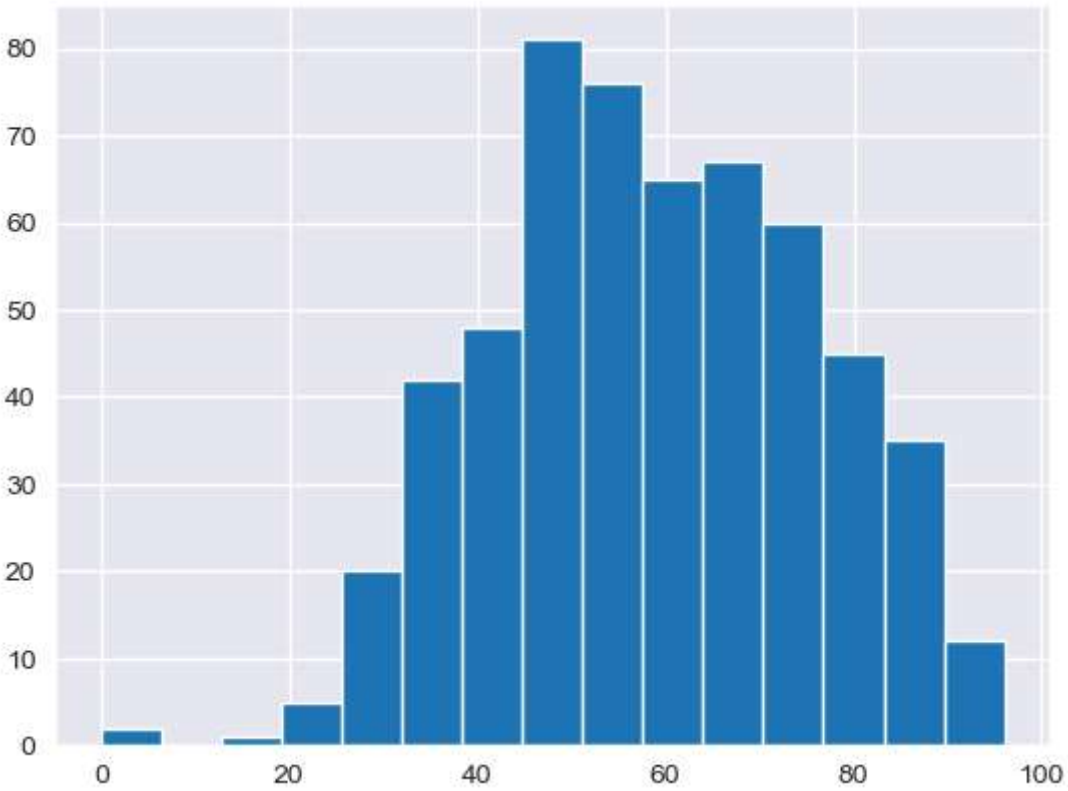
In [31]:
```
m2 = sns.distplot(movies.AudienceRating, bins = 15)
```

In [32]:
```
plt.show()
```



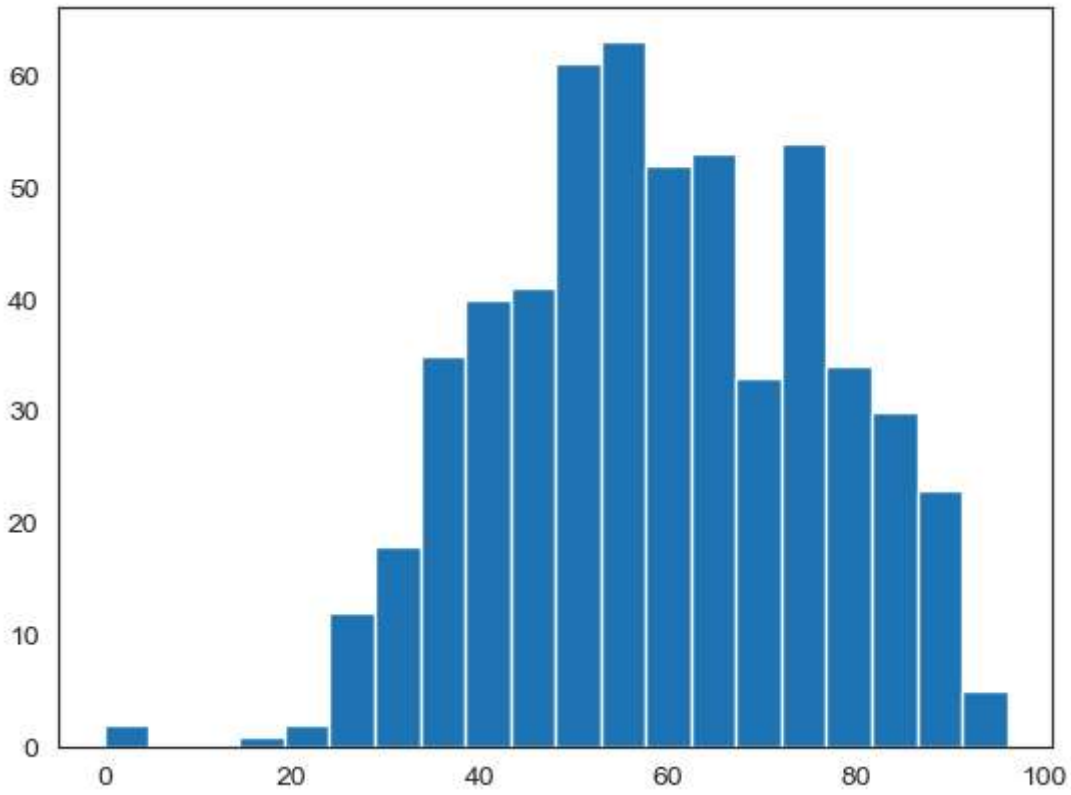
In [33]:
```
n1 = plt.hist(movies.AudienceRating, bins=15)
```

In [34]:
```
plt.show()
```



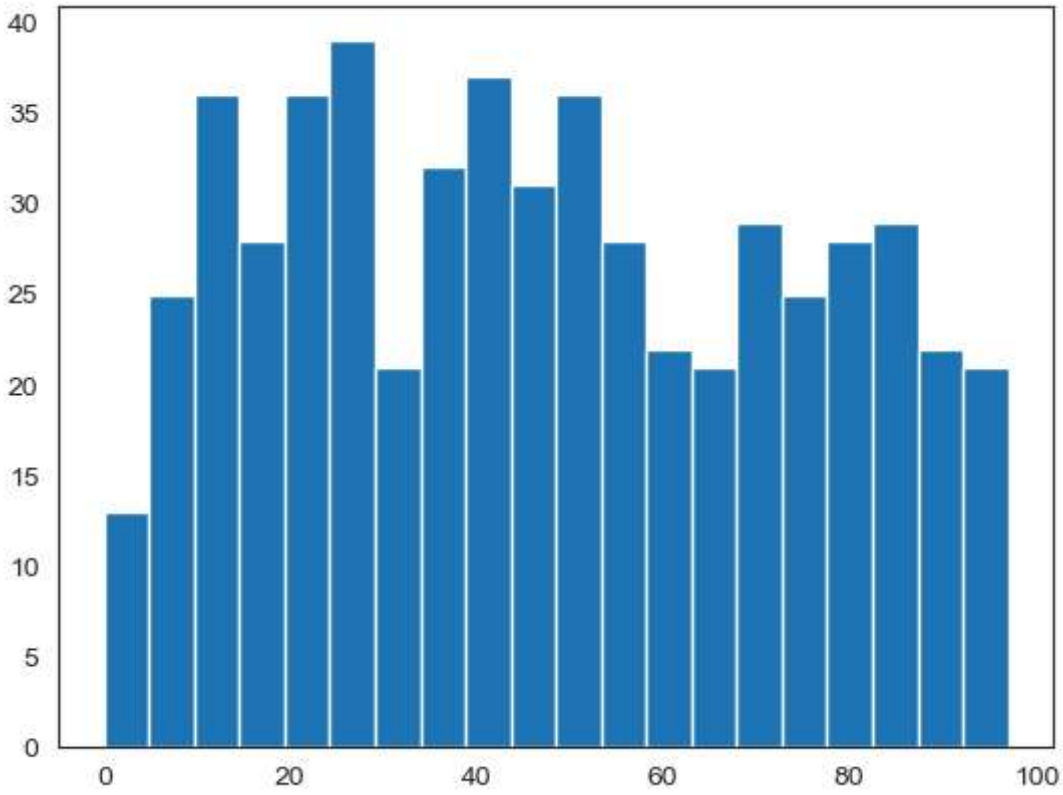
In [35]:
```
sns.set_style('white') #normal distribution & called as bell curve
n1 = plt.hist(movies.AudienceRating, bins=20)
```

In [36]:
```
plt.show()
```



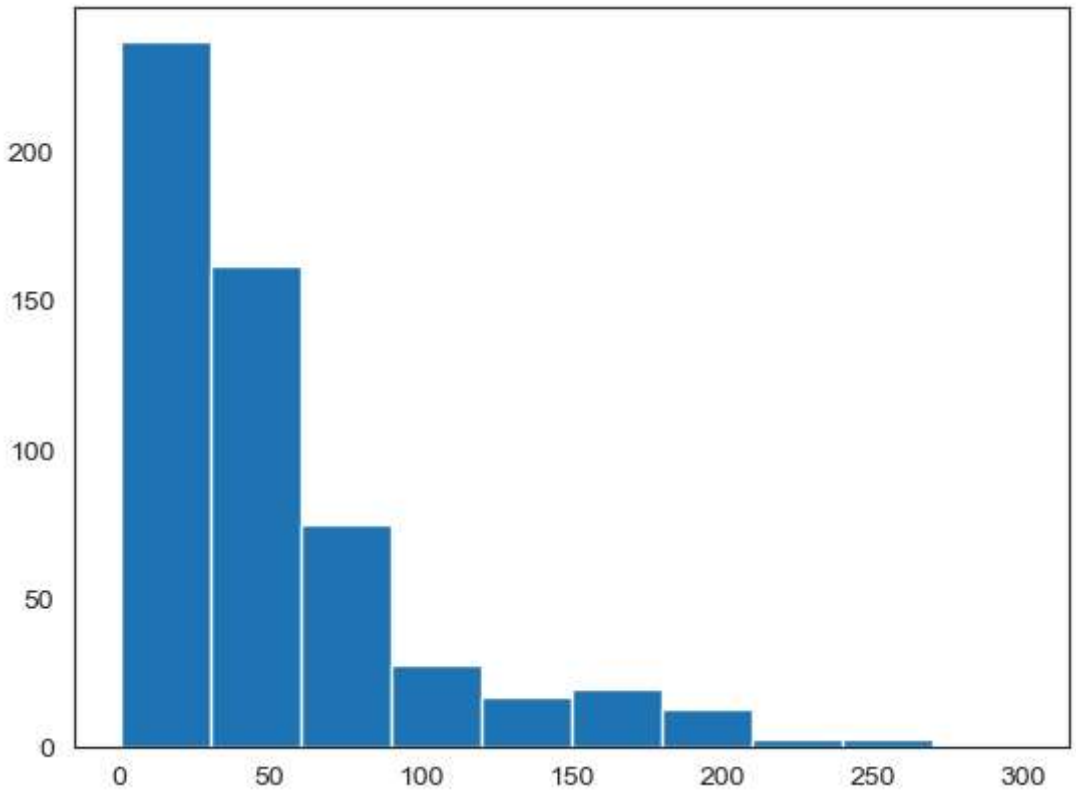
In [37]:
```
n1 = plt.hist(movies.CriticRating, bins=20) #uniform distribution
```

In [38]:
```
plt.show()
```

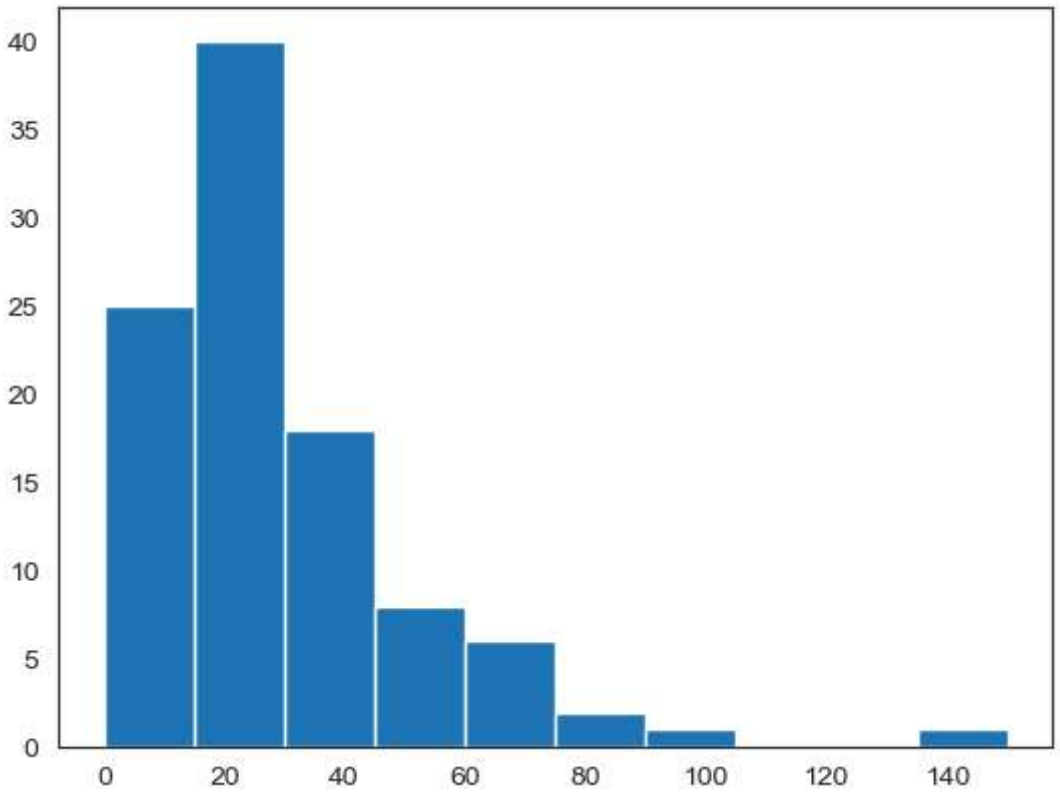


In [ ]:

In [39]:
```
plt.hist(movies.BudgetMillions)
plt.show()
```



In [40]:
```
plt.hist(movies[movies.Genre == 'Drama'].BudgetMillions)
plt.show()
```

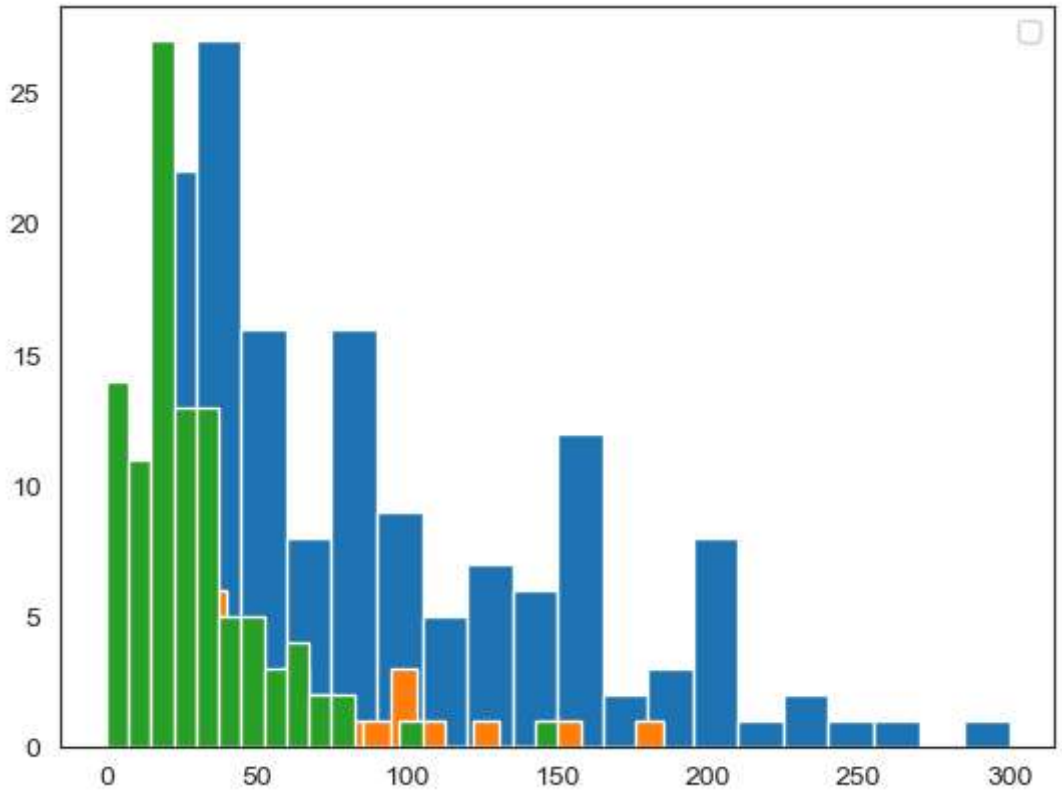


In [41]:
```
movies.head()
```

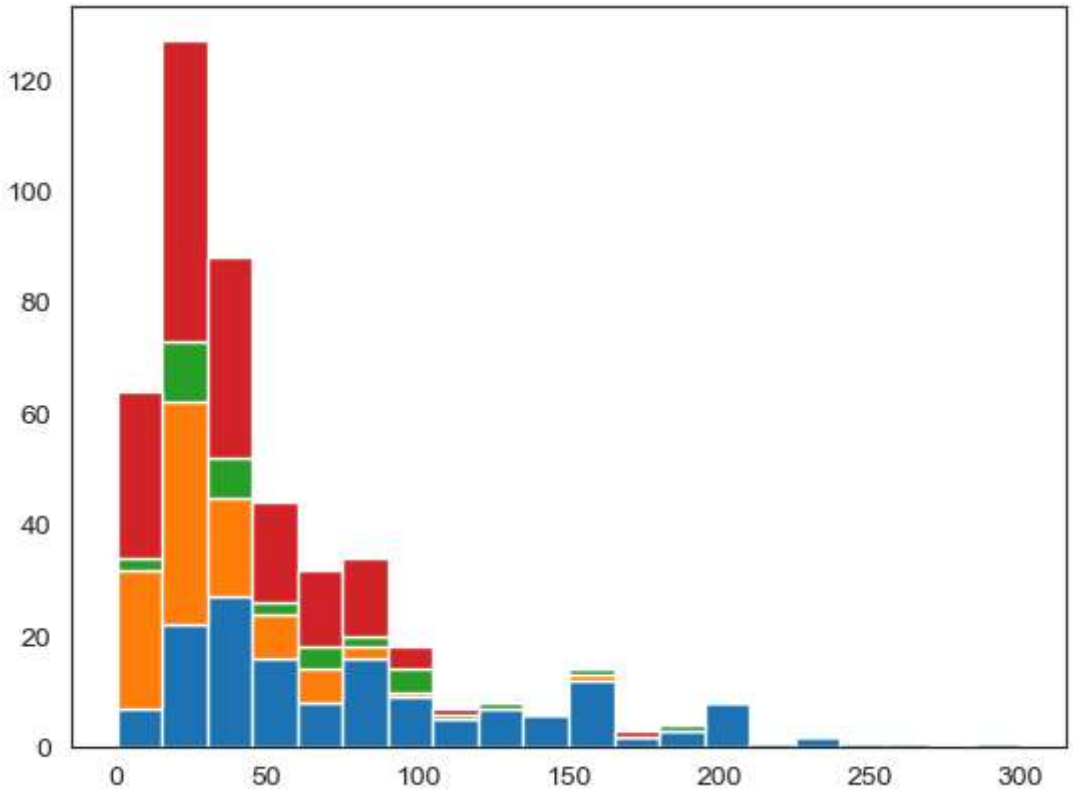
Out[41]:

	Film	Genre	CriticRating	AudienceRating	BudgetMillions	Year
0	(500) Days of Summer	Comedy	87	81	8	2009
1	10,000 B.C.	Adventure	9	44	105	2008
2	12 Rounds	Action	30	52	20	2009
3	127 Hours	Adventure	93	84	18	2010
4	17 Again	Comedy	55	70	20	2009

In [42]:
```
plt.hist(movies[movies.Genre == 'Action'].BudgetMillions, bins = 20)
plt.hist(movies[movies.Genre == 'Thriller'].BudgetMillions, bins = 20)
plt.hist(movies[movies.Genre == 'Drama'].BudgetMillions, bins = 20)
plt.legend()
plt.show()
```



In [43]:
```
plt.hist([movies[movies.Genre == 'Action'].BudgetMillions,\
          movies[movies.Genre == 'Drama'].BudgetMillions, \
          movies[movies.Genre == 'Thriller'].BudgetMillions, \
          movies[movies.Genre == 'Comedy'].BudgetMillions],
         bins = 20, stacked = True)
plt.show()
```

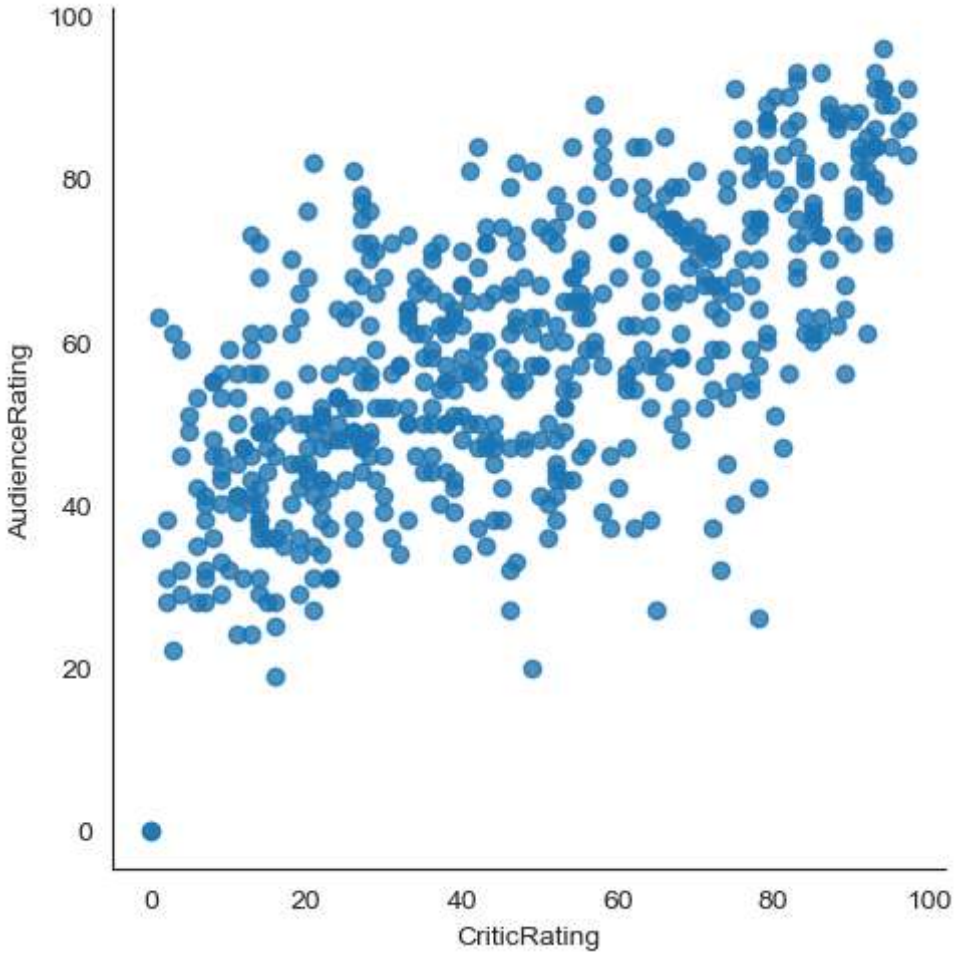


In [44]:
```
for gen in movies.Genre.cat.categories:
    print(gen)
```

```
Action
Adventure
Comedy
Drama
Horror
Romance
Thriller
```

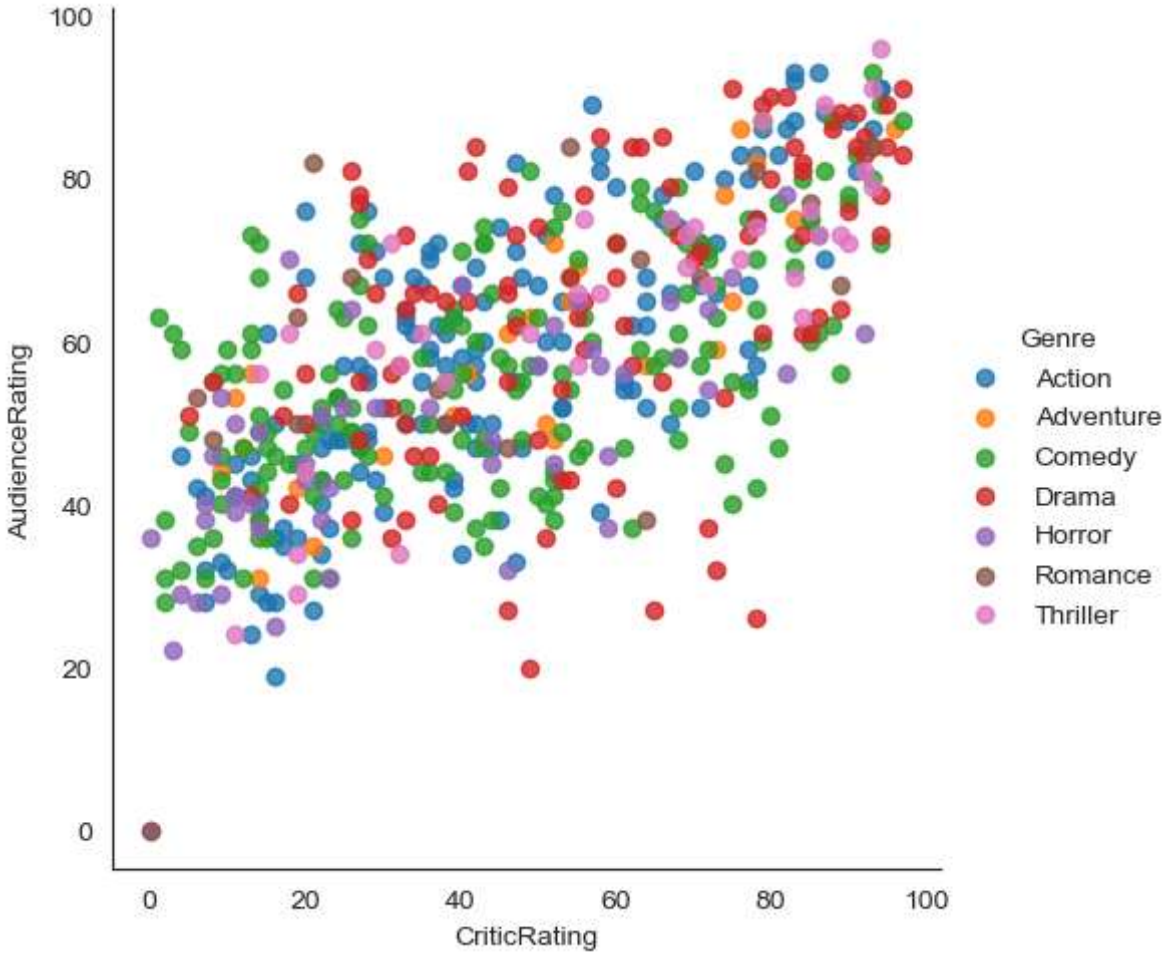
In [45]:
```
vis1 = sns.lmplot(data=movies, x='CriticRating', y='AudienceRating',\
                  fit_reg=False)
```

In [46]:
```
plt.show()
```



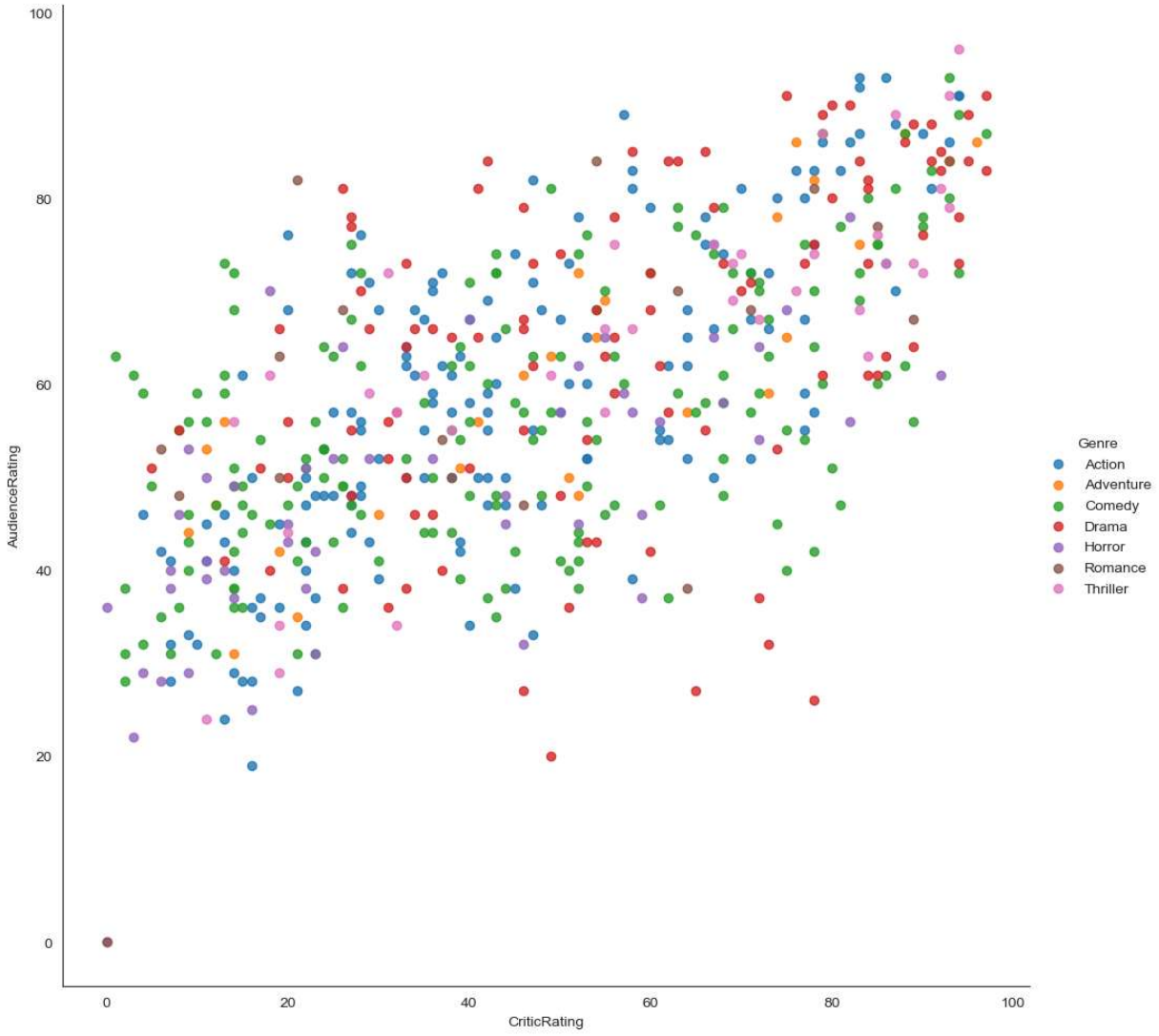
In [47]:
```
vis1 = sns.lmplot(data=movies, x='CriticRating', y='AudienceRating',\
                  fit_reg=False, hue = 'Genre')
```

In [48]:
```
plt.show()
```

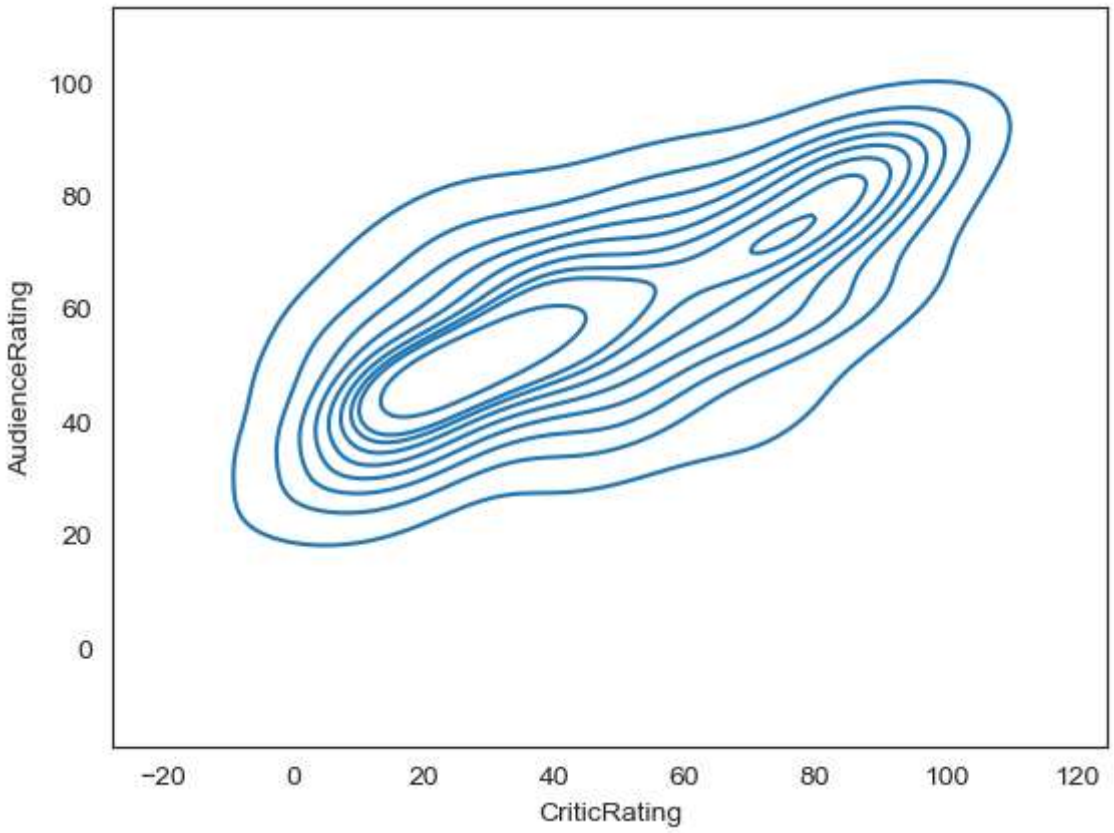


In [49]:
```
vis1 = sns.lmplot(data=movies, x='CriticRating', y='AudienceRating', \
                  fit_reg=False, hue='Genre', height=10, aspect=1)
```

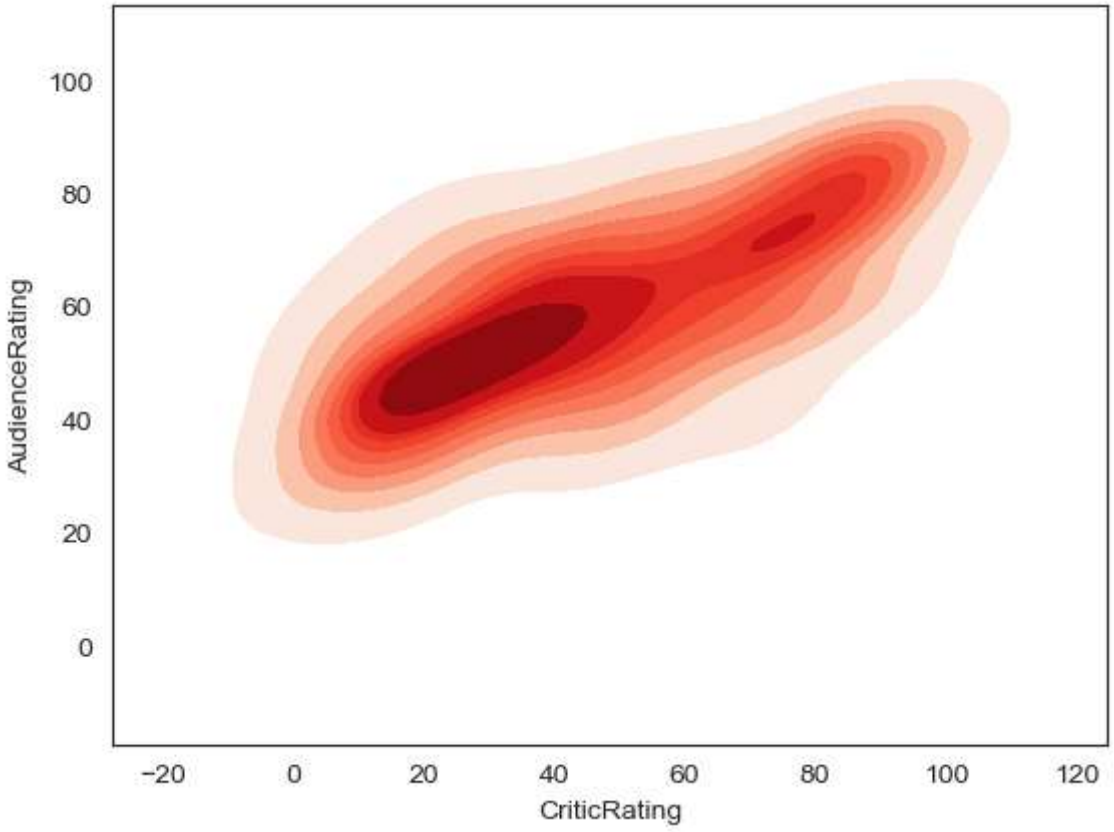
In [50]:
```
plt.show()
```



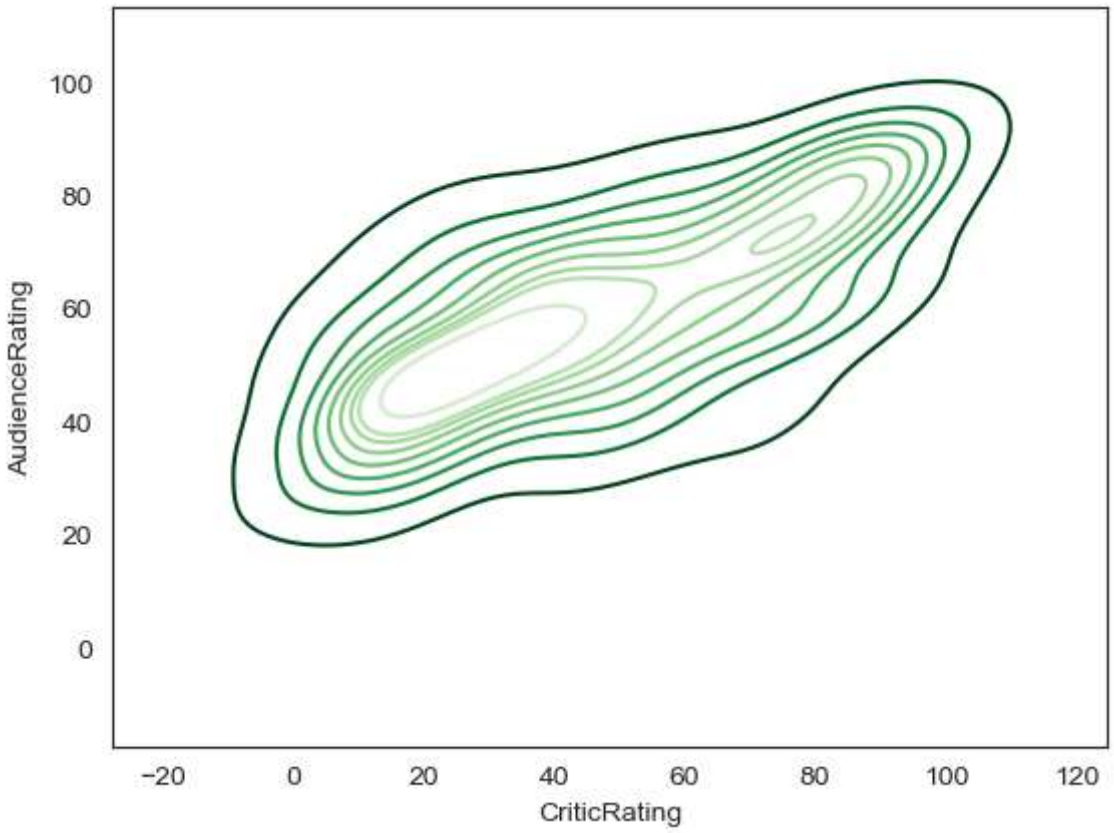
In [52]:
```
k1 = sns.kdeplot(x=movies.CriticRating, y=movies.AudienceRating)
plt.show()
```



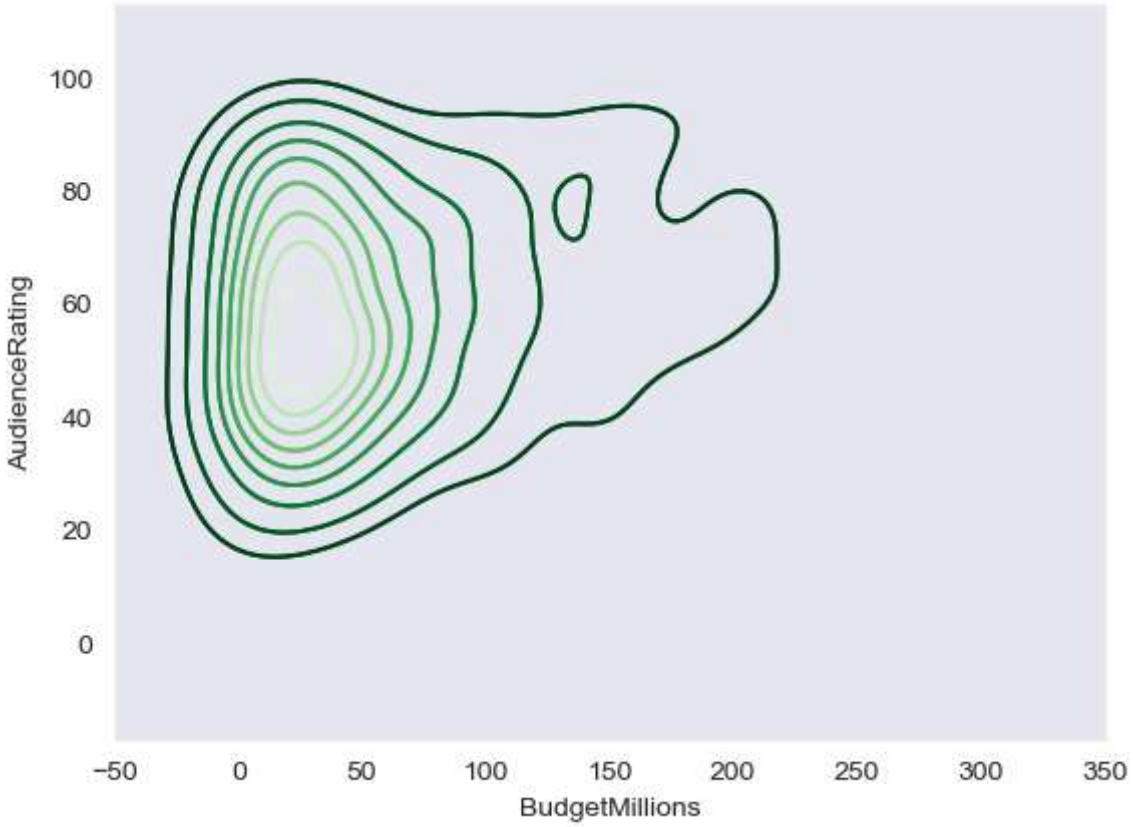
In [54]:
```
k1 = sns.kdeplot(x=movies.CriticRating, y=movies.AudienceRating,shade = True,shade_
plt.show()
```



In [56]:
```
k1 = sns.kdeplot(x=movies.CriticRating, y=movies.AudienceRating,shade_lowest=False,
plt.show()
```

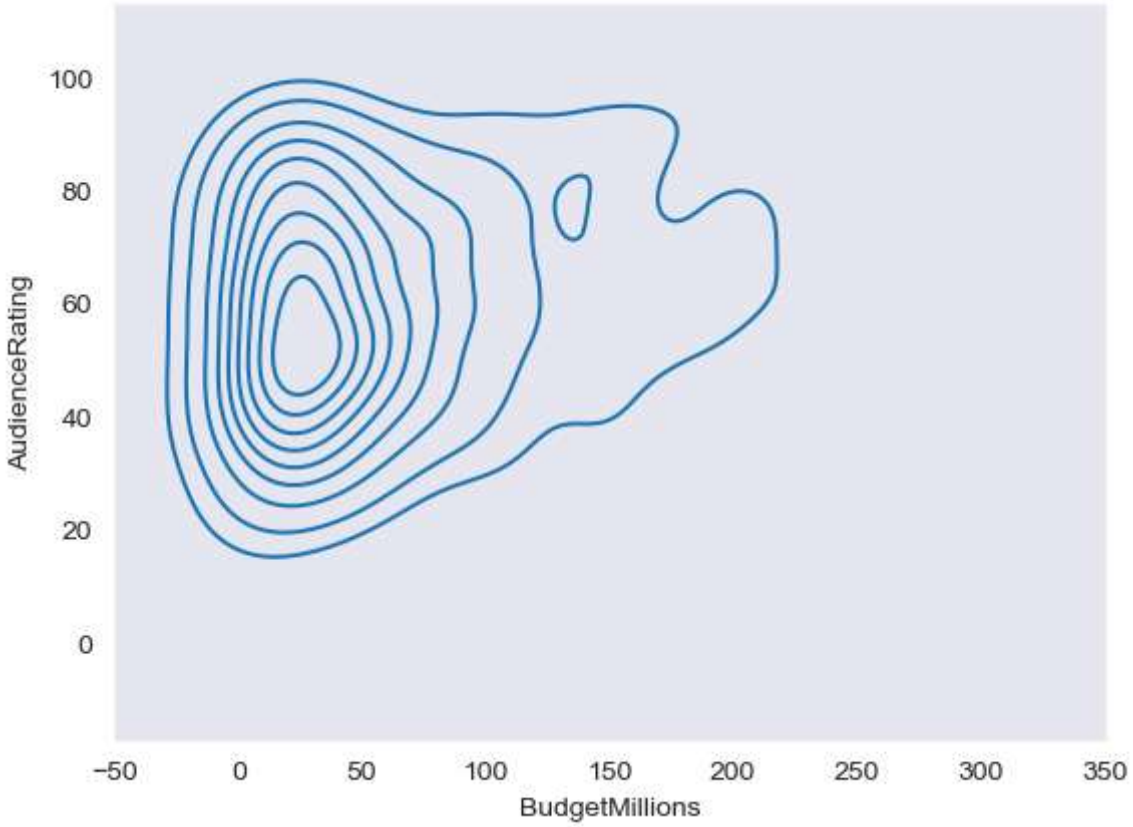


In [59]:
```
sns.set_style('dark')
k1 = sns.kdeplot(x=movies.BudgetMillions, y=movies.AudienceRating,shade_lowest=Fals
plt.show()
```



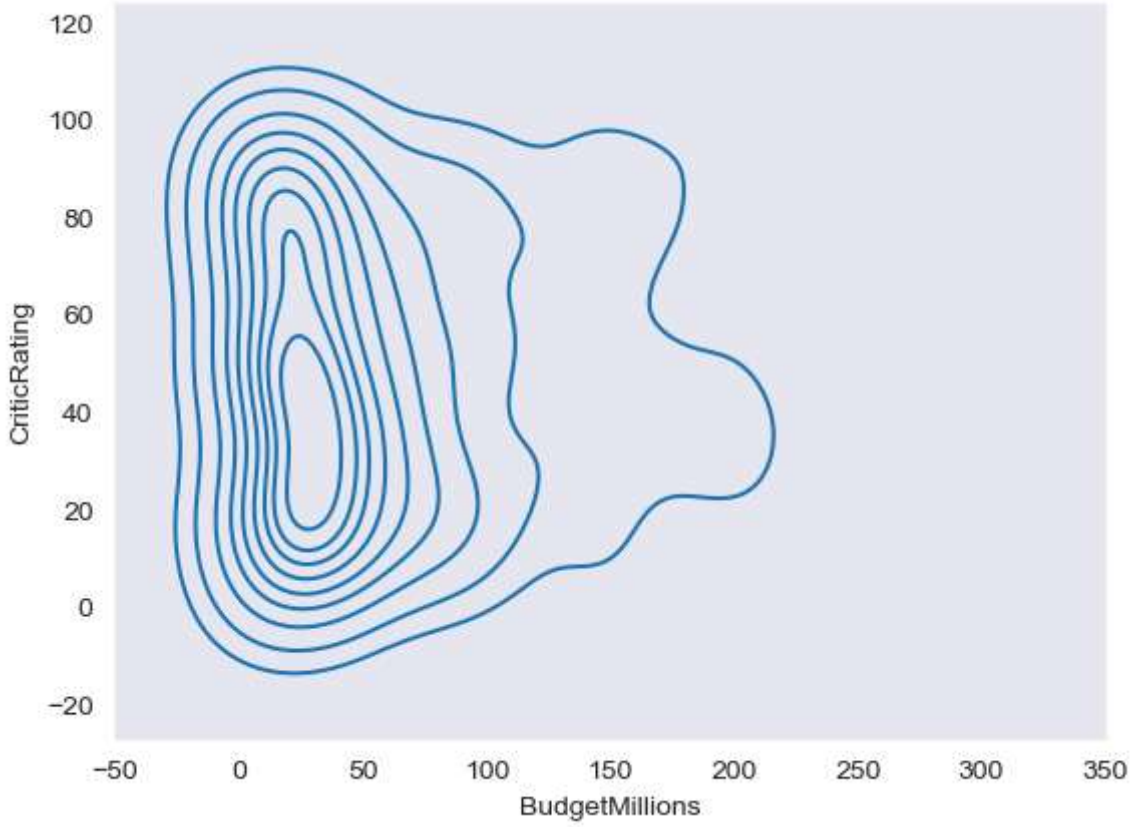
In [60]:
```
sns.set_style('dark')
k1 = sns.kdeplot(x=movies.BudgetMillions,y=movies.AudienceRating)
```

In [61]:
```
plt.show()
```



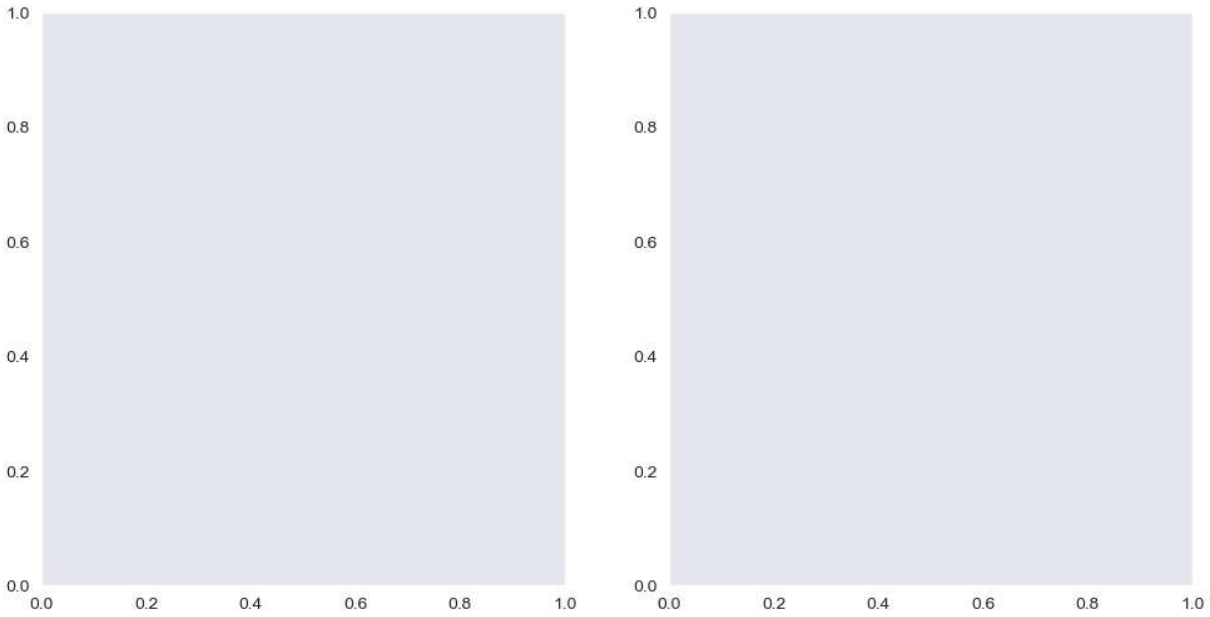
In [63]:
```
k2 = sns.kdeplot(x=movies.BudgetMillions,y=movies.CriticRating)
```

In [64]:
```
plt.show()
```



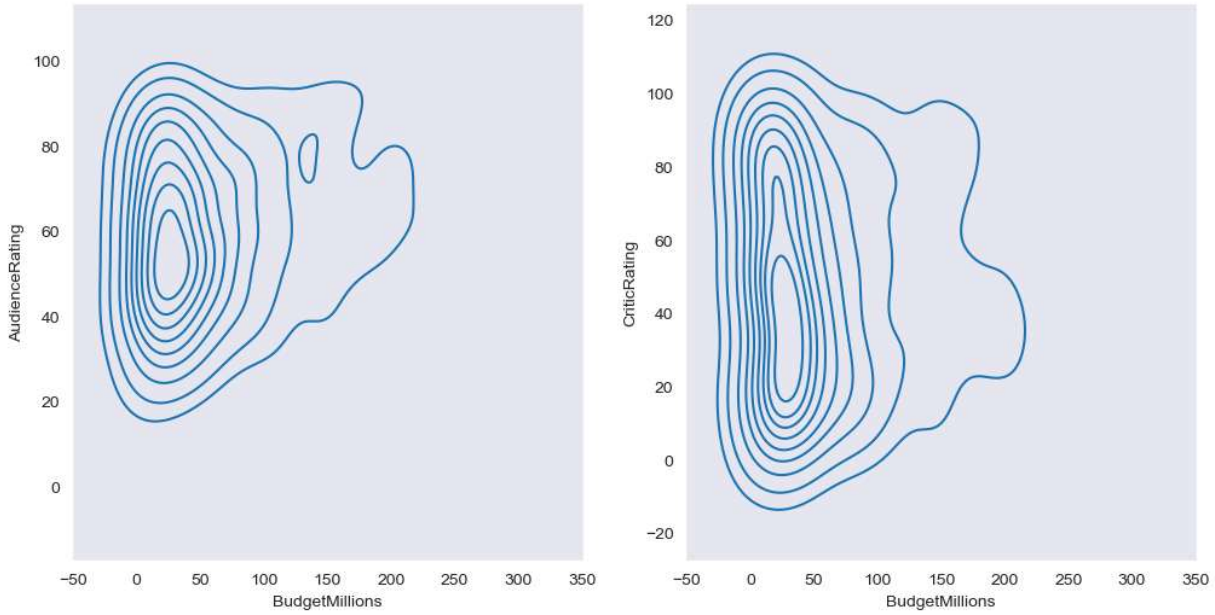
In [65]:
```
f, ax = plt.subplots(1,2, figsize =(12,6))
```

In [66]:
```
plt.show()
```



In [71]:
```
f, axes = plt.subplots(1,2, figsize =(12,6))

k1 = sns.kdeplot(x=movies.BudgetMillions,y=movies.AudienceRating,ax=axes[0])
k2 = sns.kdeplot(x=movies.BudgetMillions,y=movies.CriticRating,ax = axes[1])
```

In [72]:
```
plt.show()
```

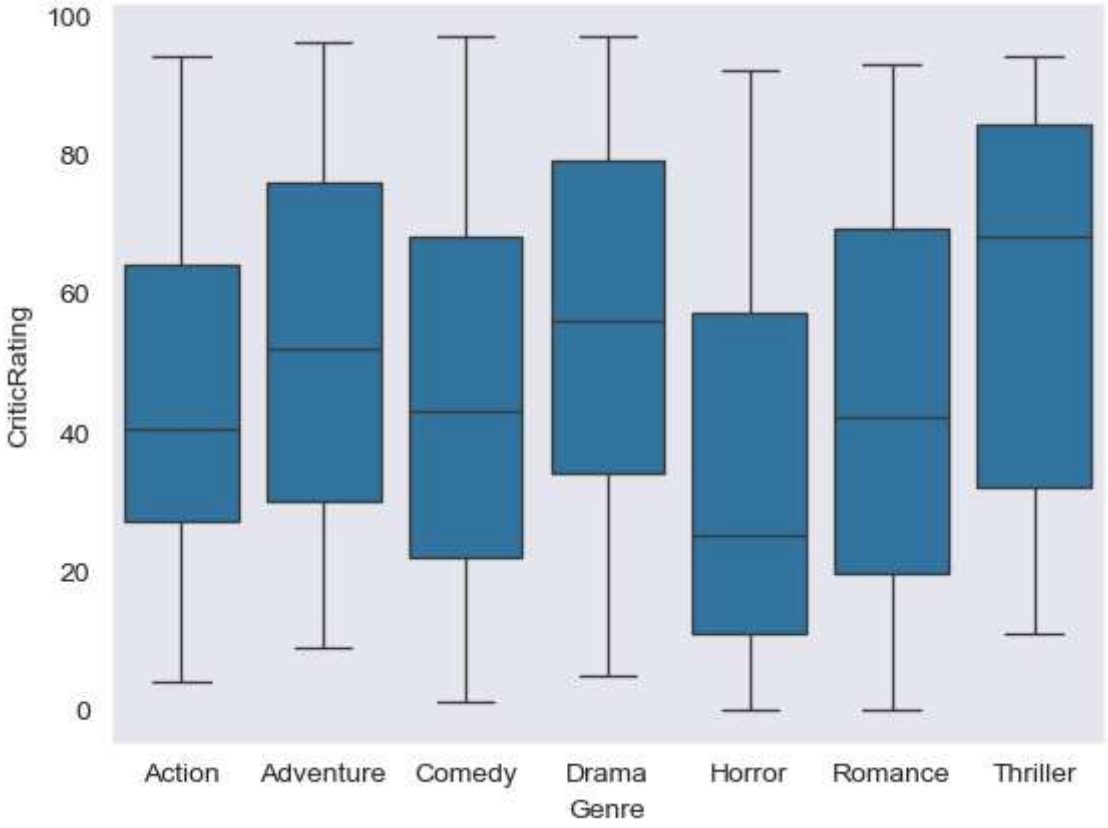


In [73]:
```
axes
```

Out[73]:
```
array([<Axes: xlabel='BudgetMillions', ylabel='AudienceRating'>,
       <Axes: xlabel='BudgetMillions', ylabel='CriticRating'>],
      dtype=object)
```

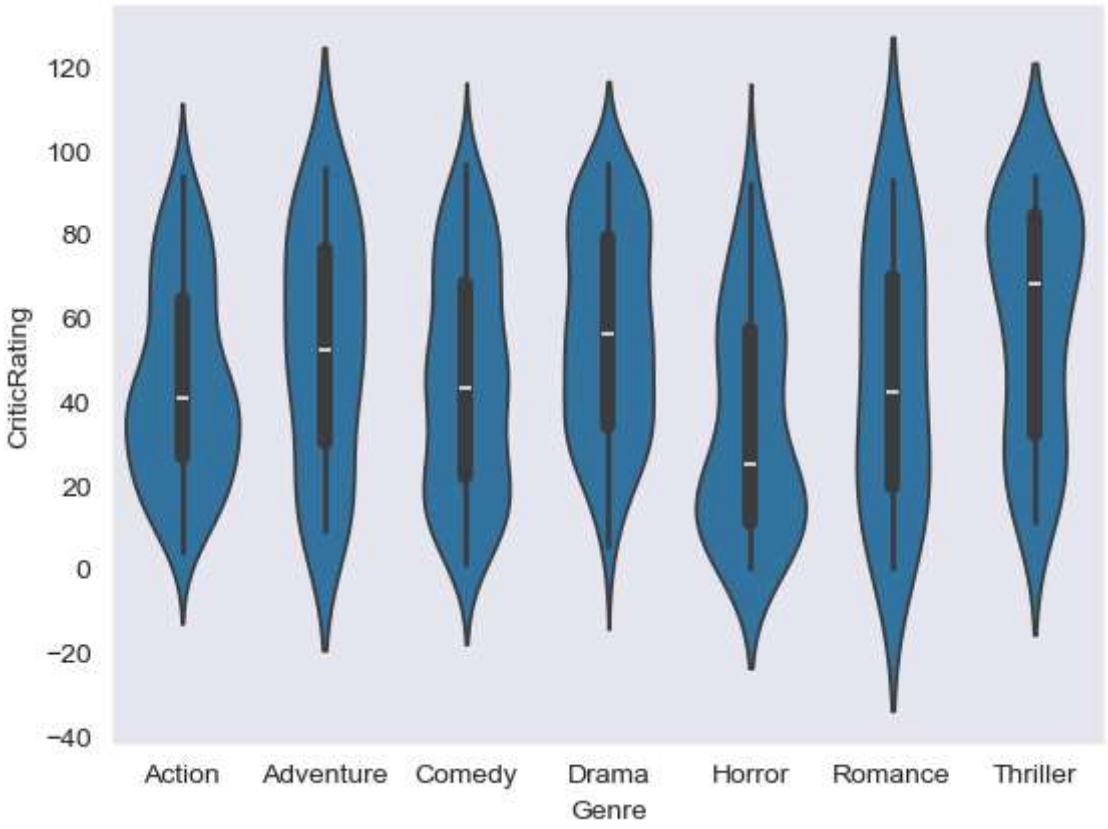
In [76]:
```
w = sns.boxplot(data=movies, x='Genre', y = 'CriticRating')
```

In [77]:
```
plt.show()
```



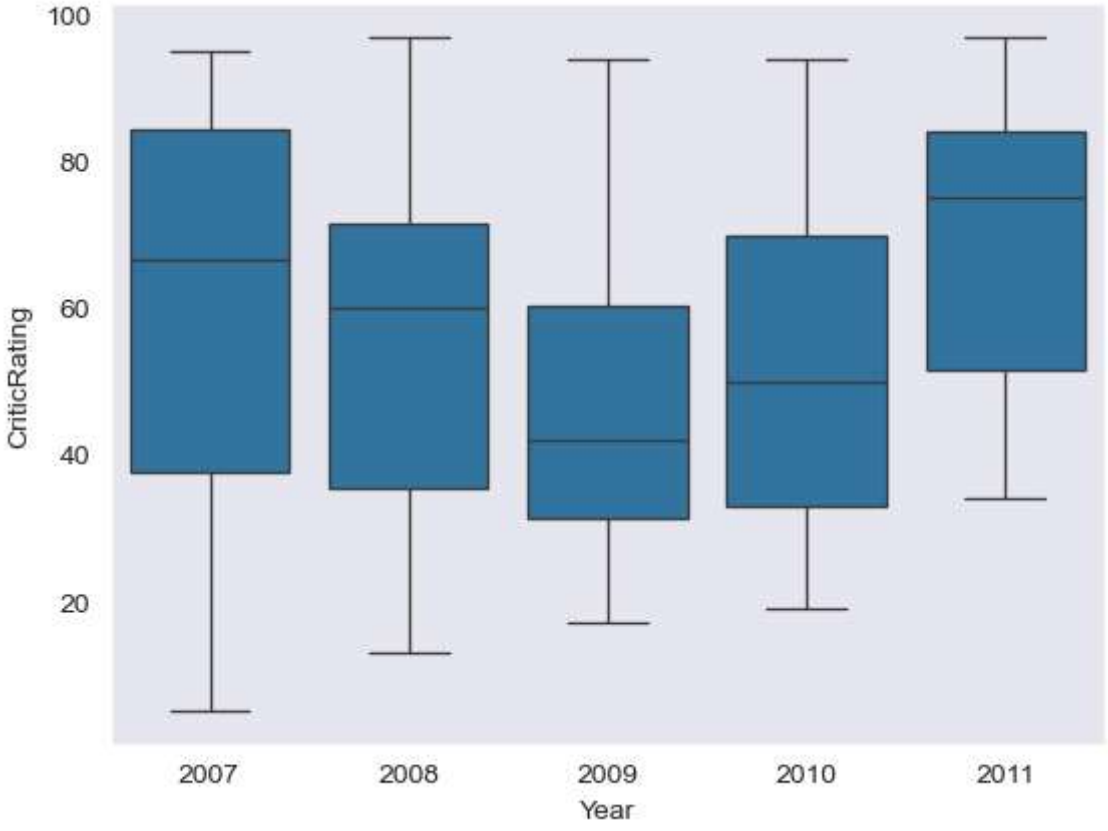
In [78]:
```
z = sns.violinplot(data=movies, x='Genre', y = 'CriticRating')
```

In [79]:
```
plt.show()
```



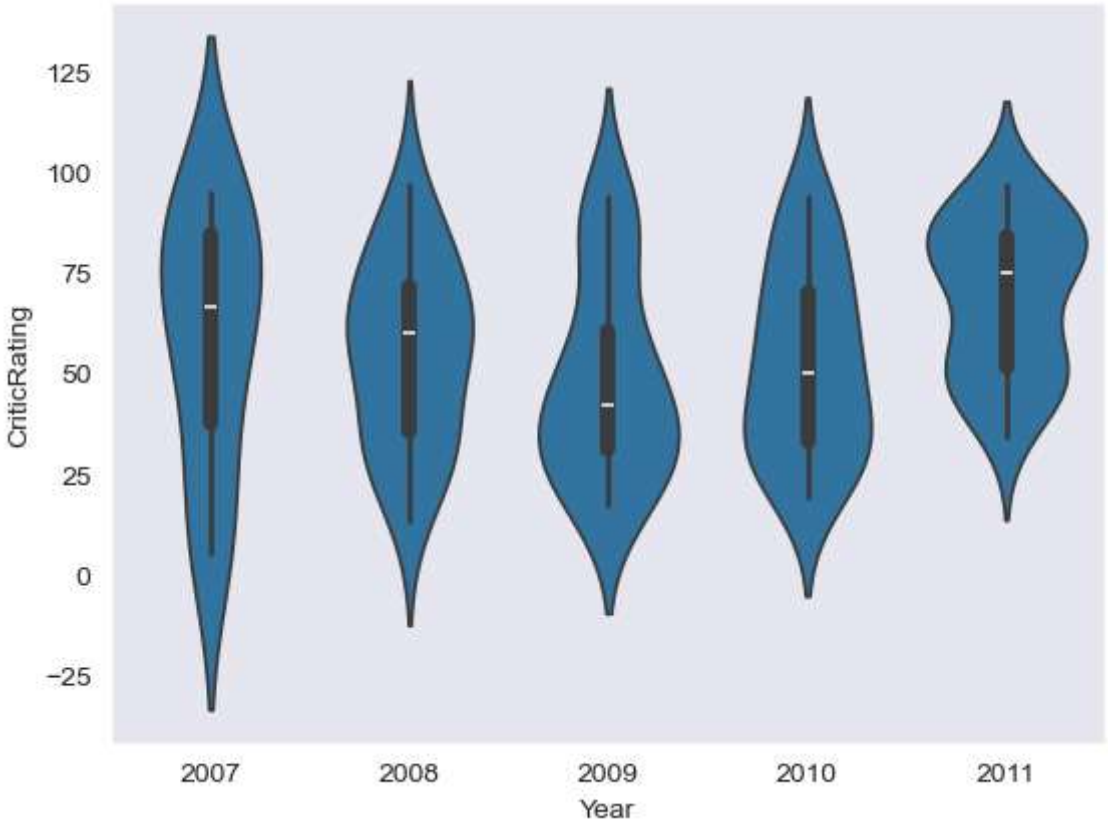
In [80]:
```
w1 = sns.boxplot(data=movies[movies.Genre == 'Drama'], x='Year', y = 'CriticRating'
```

In [81]:
```
plt.show()
```

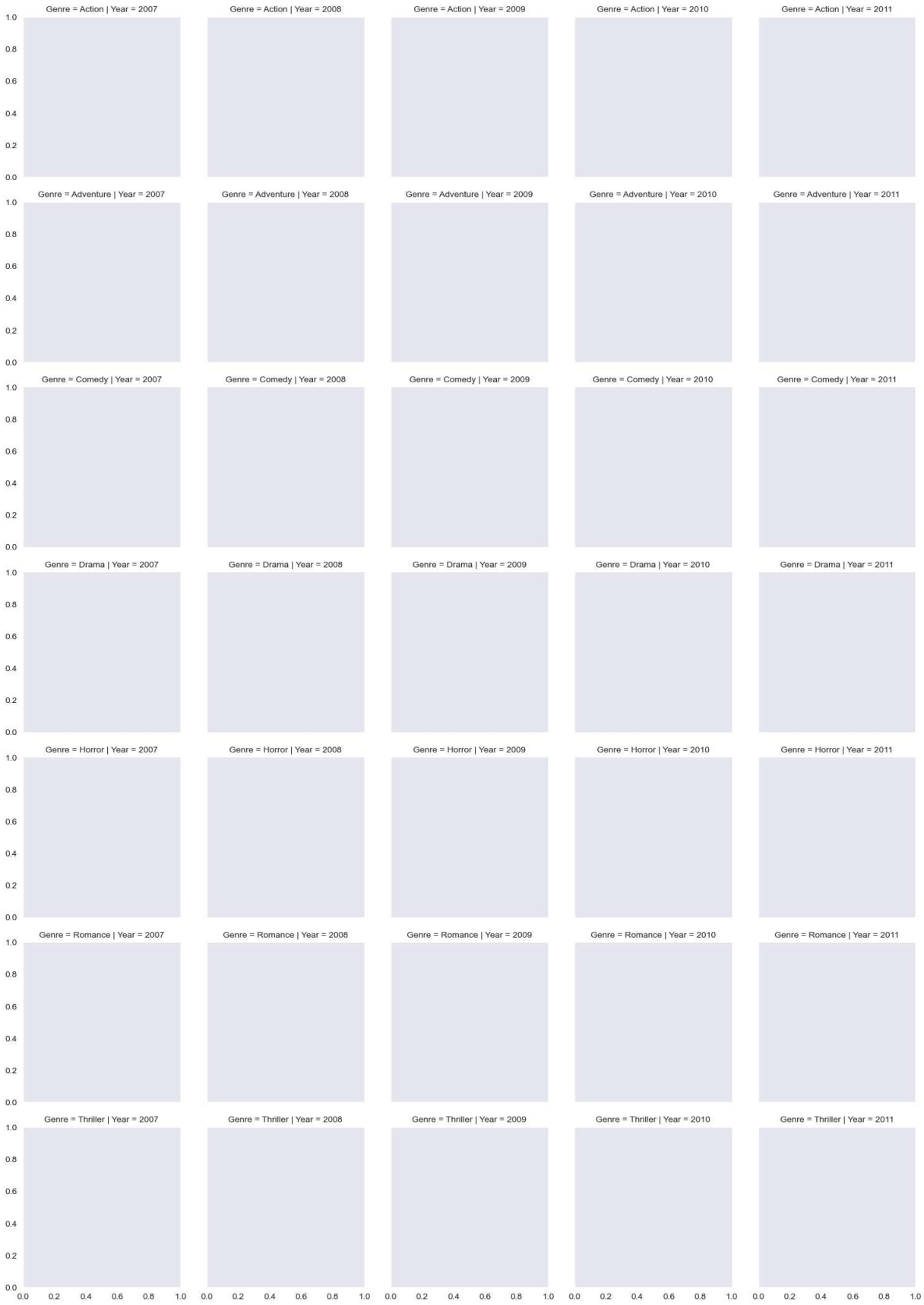


In [82]:
```
z = sns.violinplot(data=movies[movies.Genre == 'Drama'], x='Year', y = 'CriticRatin
```

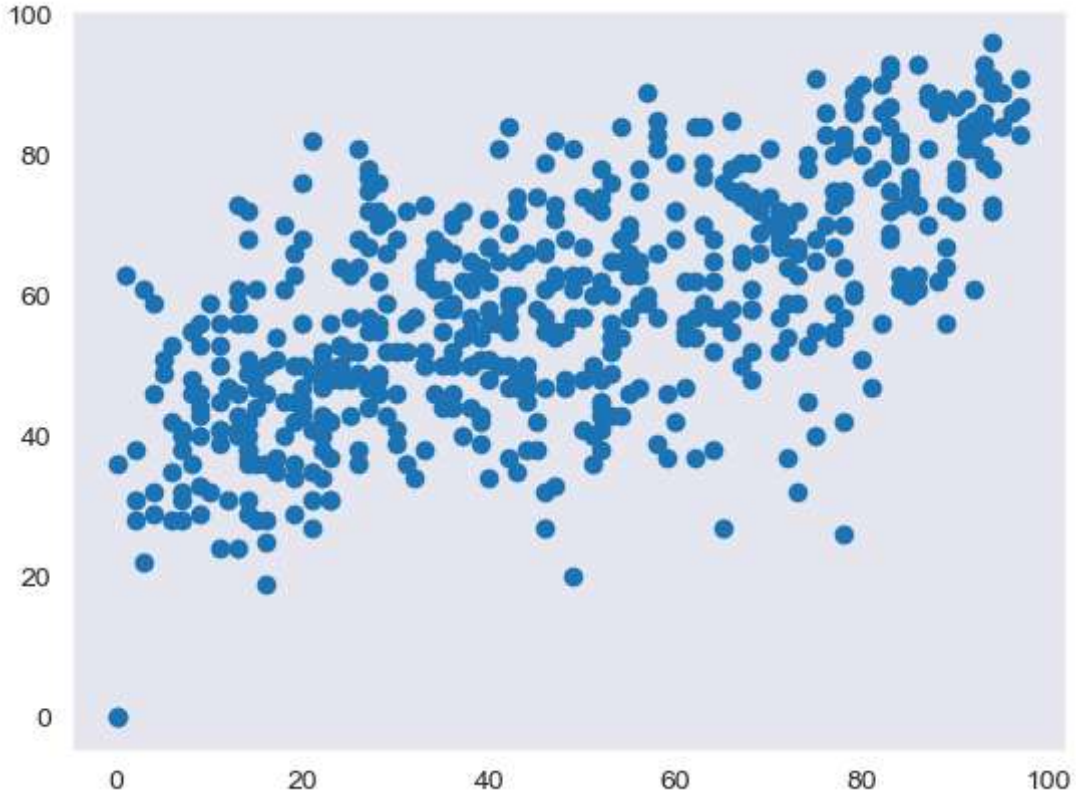
In [83]:
```
plt.show()
```



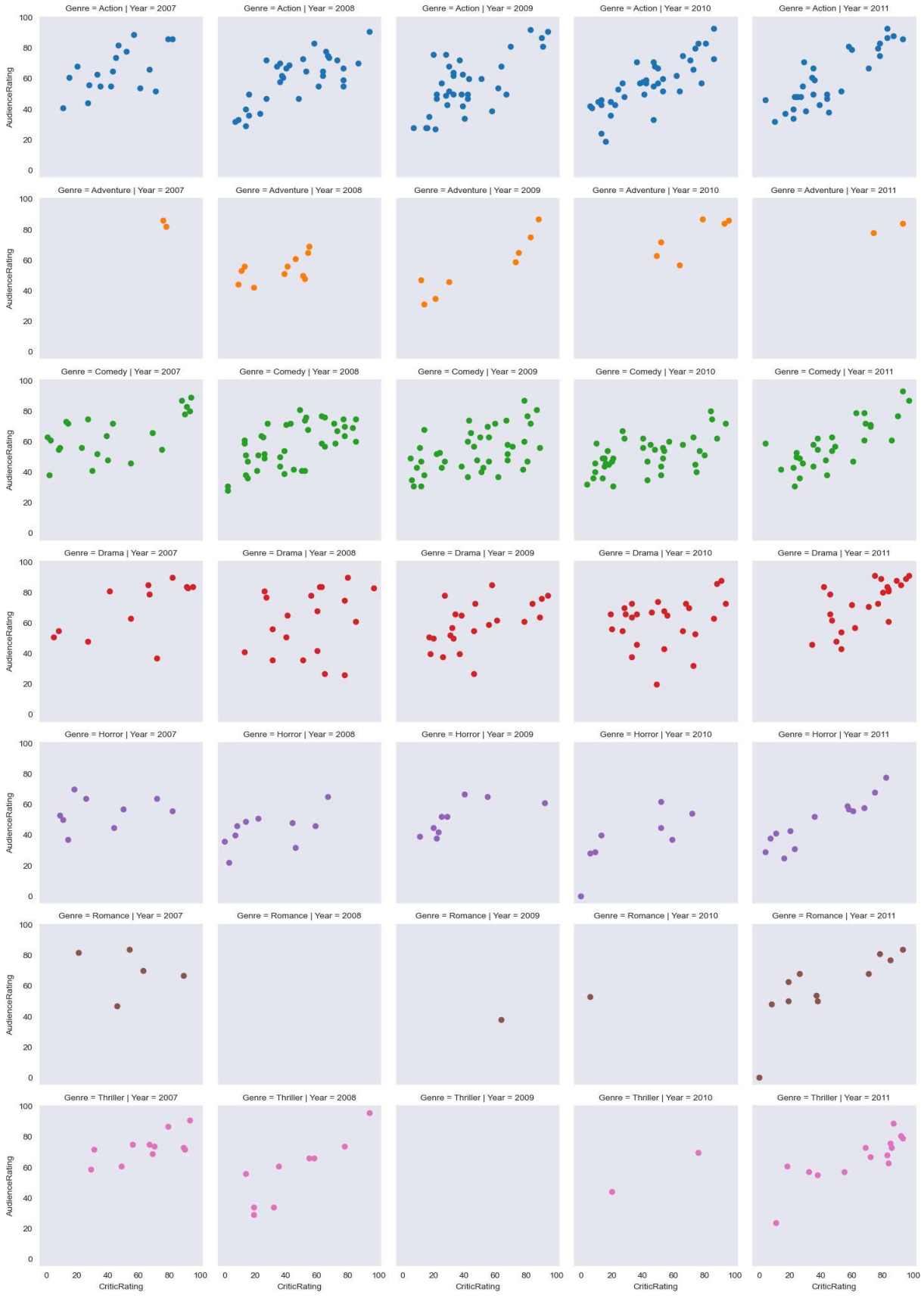
In [84]:
```
g =sns.FacetGrid (movies, row = 'Genre', col = 'Year', hue = 'Genre') #kind of subp
plt.show()
```



In [85]:
```
plt.scatter(movies.CriticRating,movies.AudienceRating)
plt.show()
```

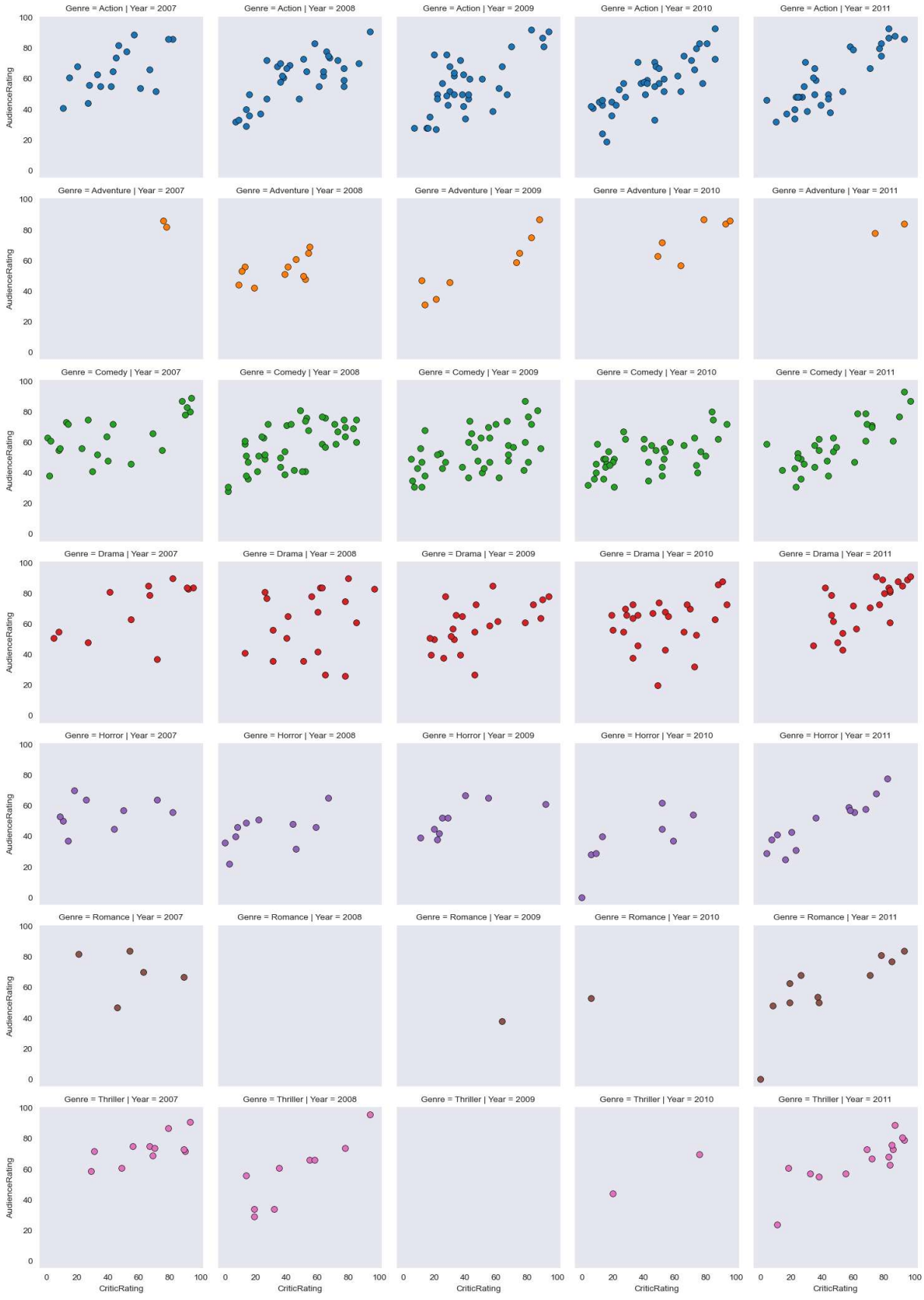


In [86]:
```
g =sns.FacetGrid (movies, row = 'Genre', col = 'Year', hue = 'Genre')
g = g.map(plt.scatter, 'CriticRating', 'AudienceRating' )
plt.show()
```



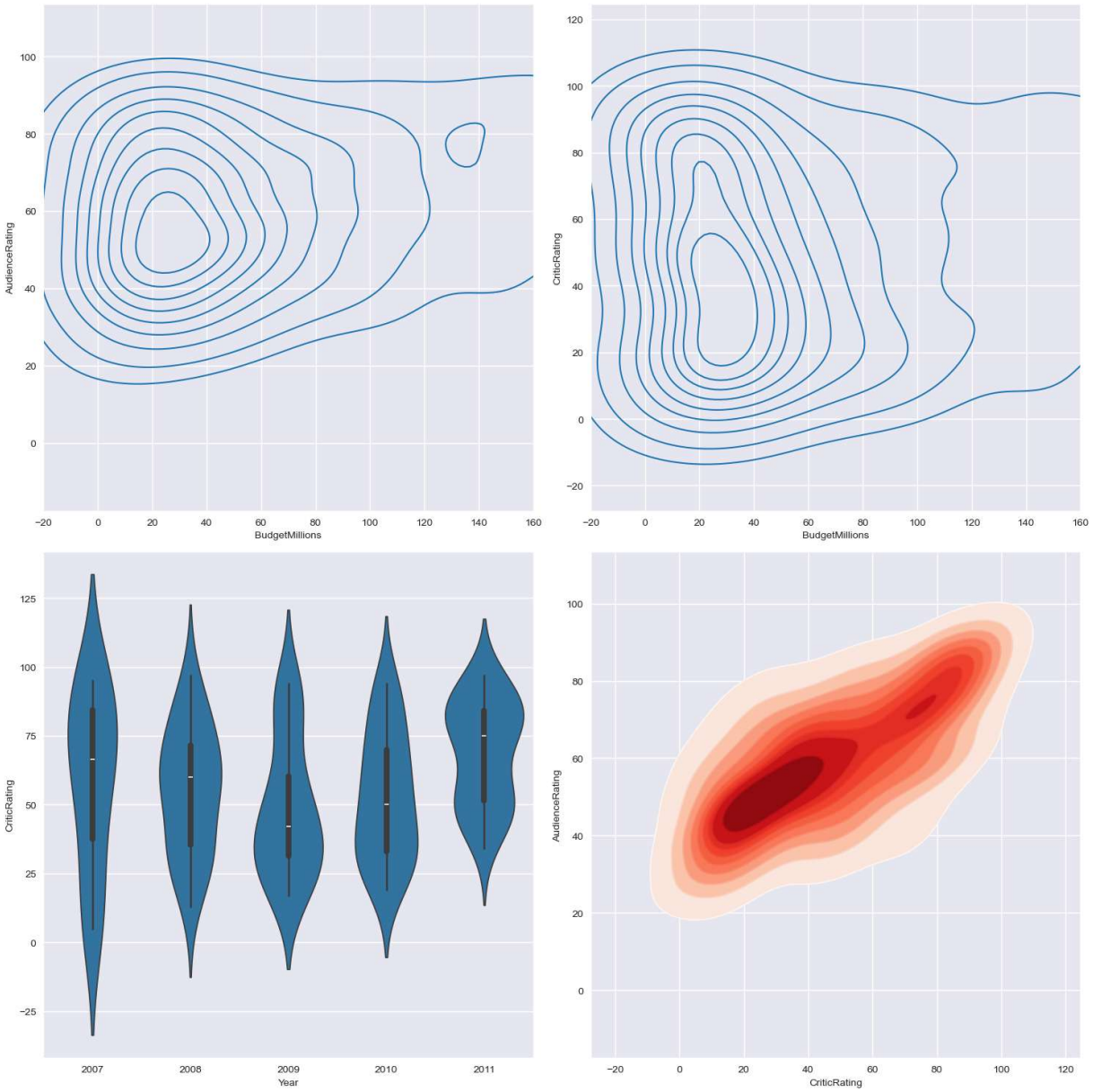
In [87]:
```
g =sns.FacetGrid (movies, row = 'Genre', col = 'Year', hue = 'Genre')
g = g.map(plt.hist, 'BudgetMillions')
plt.show()
```

In [88]:
```
g =sns.FacetGrid (movies, row = 'Genre', col = 'Year', hue = 'Genre')
kws = dict(s=50, linewidth=0.5,edgecolor='black')
g = g.map(plt.scatter, 'CriticRating', 'AudienceRating',**kws )
plt.show()
```



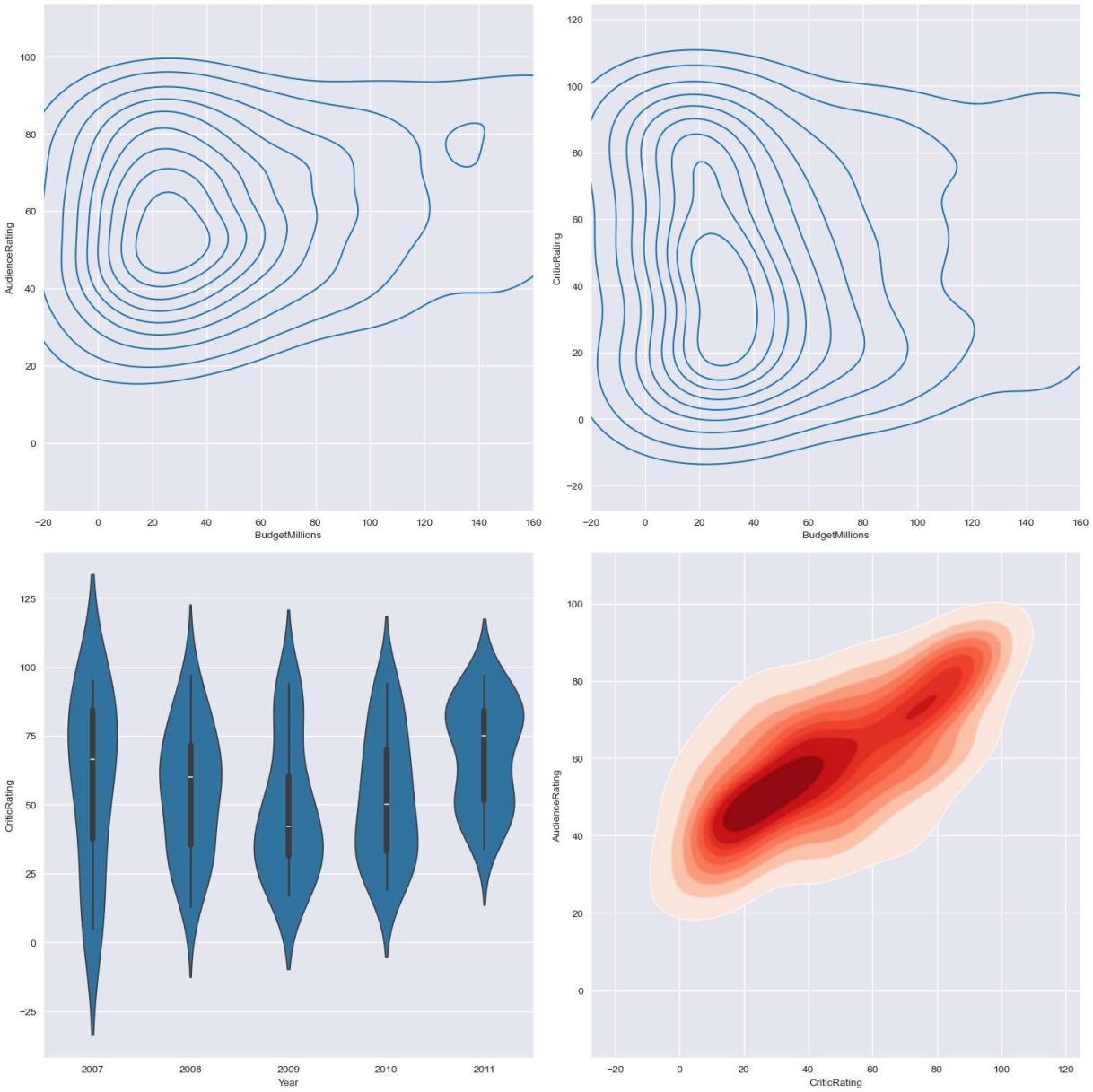
In [91]:
```
sns.set_style('darkgrid')
f, axes = plt.subplots(2, 2, figsize=(15, 15))

k1 = sns.kdeplot(x=movies.BudgetMillions, y=movies.AudienceRating, ax=axes[0, 0])
```

```
k2 = sns.kdeplot(x=movies.BudgetMillions, y=movies.CriticRating, ax=axes[0, 1])

k1.set(xlim=(-20, 160))
k2.set(xlim=(-20, 160))

z = sns.violinplot(data=movies[movies.Genre == 'Drama'], x='Year', y='CriticRating'

# Corrected kdeplot calls using x and y keyword arguments
k4 = sns.kdeplot(x=movies.CriticRating, y=movies.AudienceRating, shade=True, shade_

# This line will overlay another KDE plot on the same subplot
k4b = sns.kdeplot(x=movies.CriticRating, y=movies.AudienceRating, cmap='Reds', ax=a

plt.tight_layout() # Adjust subplot params for a tight layout
plt.show()
```



In [94]:
```
sns.set_style('darkgrid')
f, axes = plt.subplots(2, 2, figsize=(15, 15))
```

```
# Corrected kdeplot calls using x and y keyword arguments
k1 = sns.kdeplot(x=movies.BudgetMillions, y=movies.AudienceRating, ax=axes[0, 0])
k2 = sns.kdeplot(x=movies.BudgetMillions, y=movies.CriticRating, ax=axes[0, 1])

k1.set(xlim=(-20, 160))
k2.set(xlim=(-20, 160))

z = sns.violinplot(data=movies[movies.Genre == 'Drama'], x='Year', y='CriticRating'

# Corrected kdeplot calls using x and y keyword arguments
k4 = sns.kdeplot(x=movies.CriticRating, y=movies.AudienceRating, shade=True, shade_

# This line will overlay another KDE plot on the same subplot
k4b = sns.kdeplot(x=movies.CriticRating, y=movies.AudienceRating, cmap='Reds', ax=a

plt.tight_layout() # Adjust subplot params for a tight layout
plt.show()
```



In [ ]: